Projet PCII

Noé BRIAND, Thanh Phat DOAN, Emilie FESQUET

1. Cahier des Charges

Le but du projet est de créer un jeu de stratégie solo en temps réel, inspiré de Factorio et Mindustry. Le but du jeu est de récolter des ressources parsemées dans la map du jeu, et de construire des bâtiments et des unités pour automatiser et accélérer la récolte de minerai (il n'y a donc pas de fin).

Le joueur commence avec un bâtiment maître et une seule unité qu'il peut diriger pour récolter du minerai. Il peut ensuite construire des mines pour récolter automatiquement, des routes pour acheminer les minerais, des bâtiments de stockage pour y mettre des ressources et des usines pour construire de nouvelles unités.

Le bâtiment maître sert initialement de stockage. Comme les autres bâtiments de stockage, il a une capacité limitée. S'il est détruit, le joueur perd la partie.

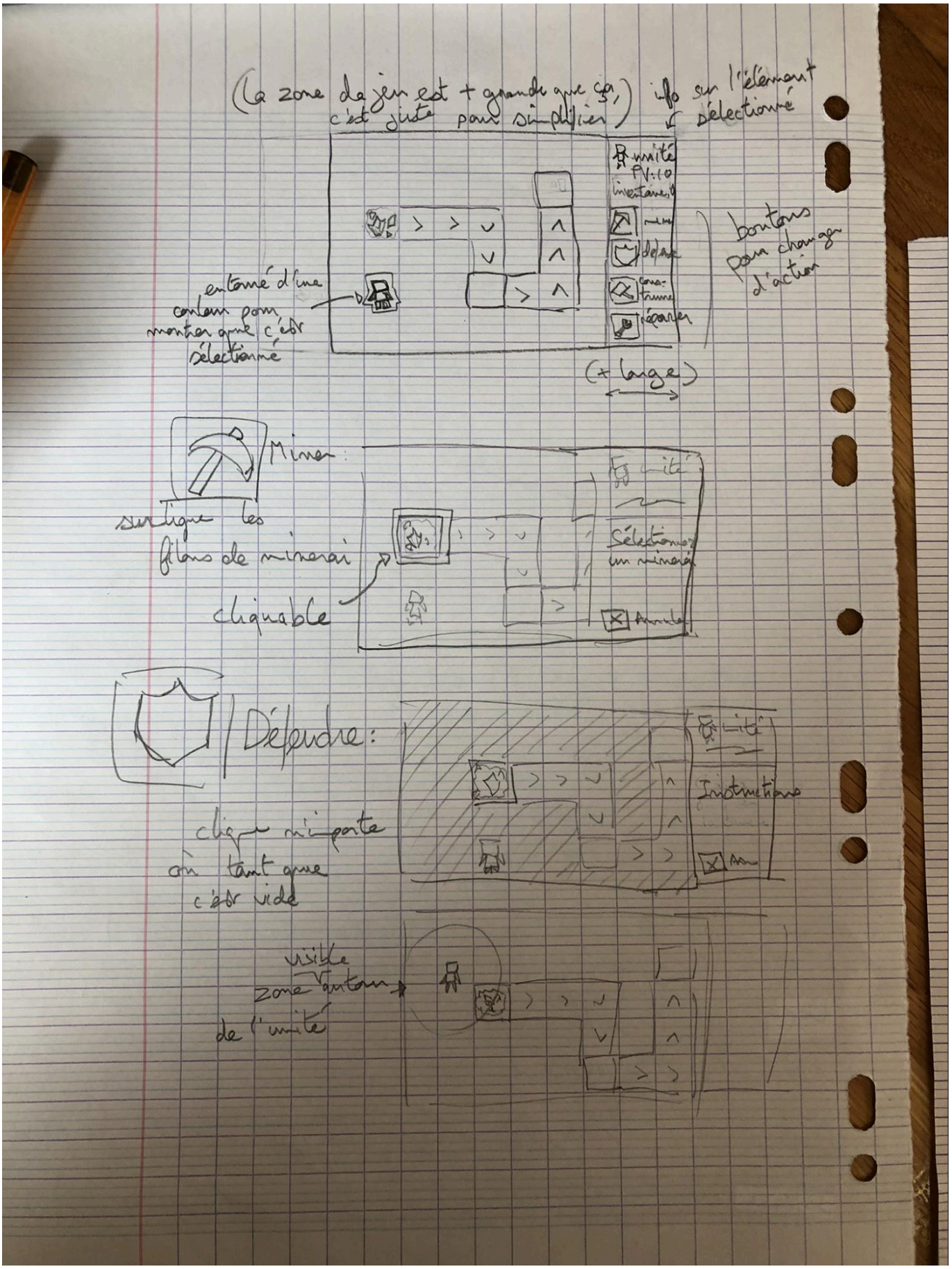
Des vagues d'ennemis arrivent régulièrement, et peuvent détruire les routes, bâtiments et unités. Les ennemis apparaissent en bordure de la carte et se déplacent en ligne droite vers le bâtiment maître. Si un bâtiment ou une unité est dans une zone autour de lui, il la détruit puis continue son chemin. S'il arrive vers une unité en mode défense, il meurt et disparaît.

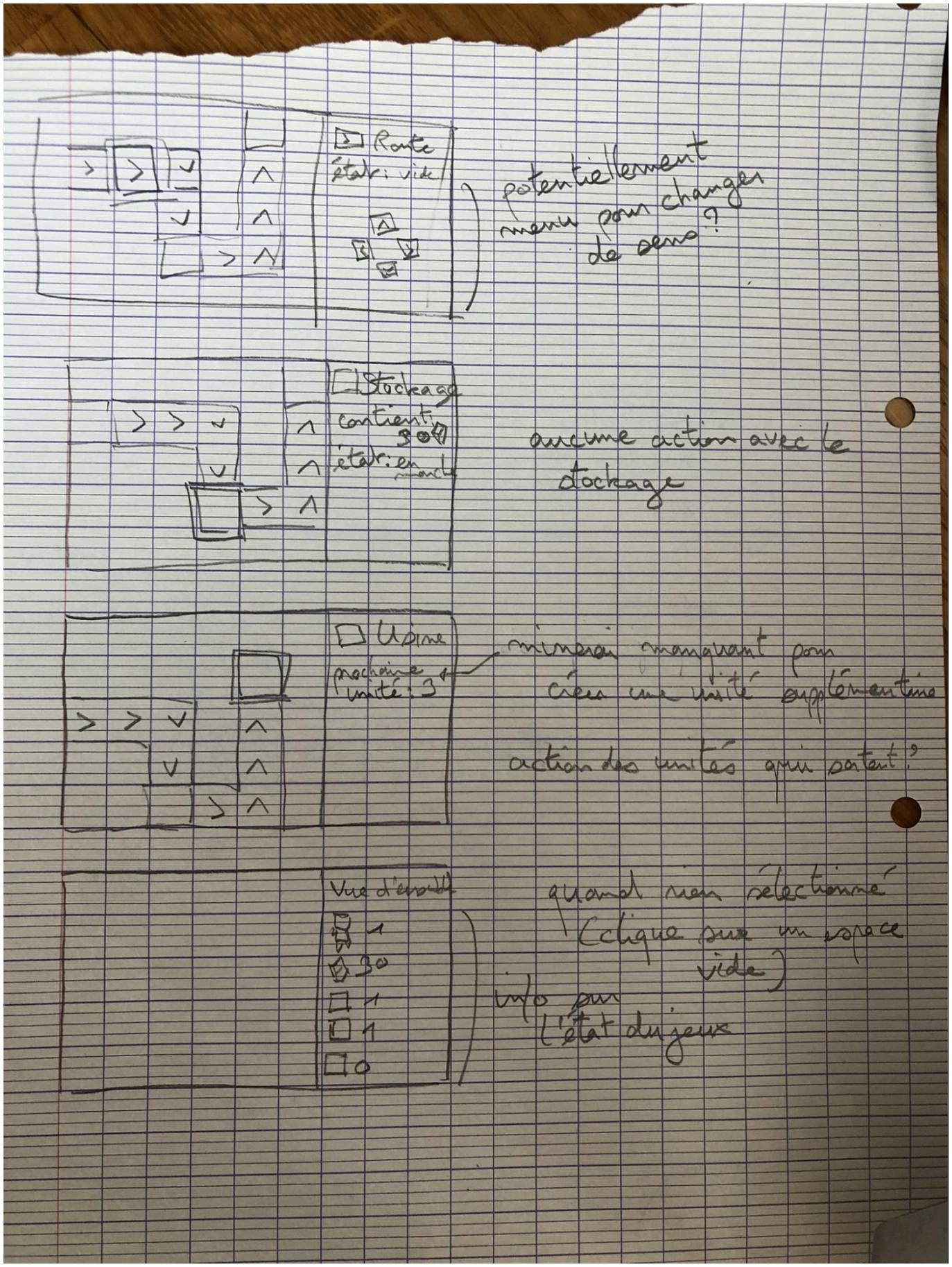
On peut ordonner aux unités de récolter du matériel, de construire des routes ou des bâtiments, ou de défendre les bâtiments existants. Les unités en mode défense se comportent comme des tourelles, et détruisent les ennemis qui entrent dans un périmètre autour d'elles. Chaque unité a un stockage local qu'elle peut utiliser pour commencer à construire des routes/bâtiments.

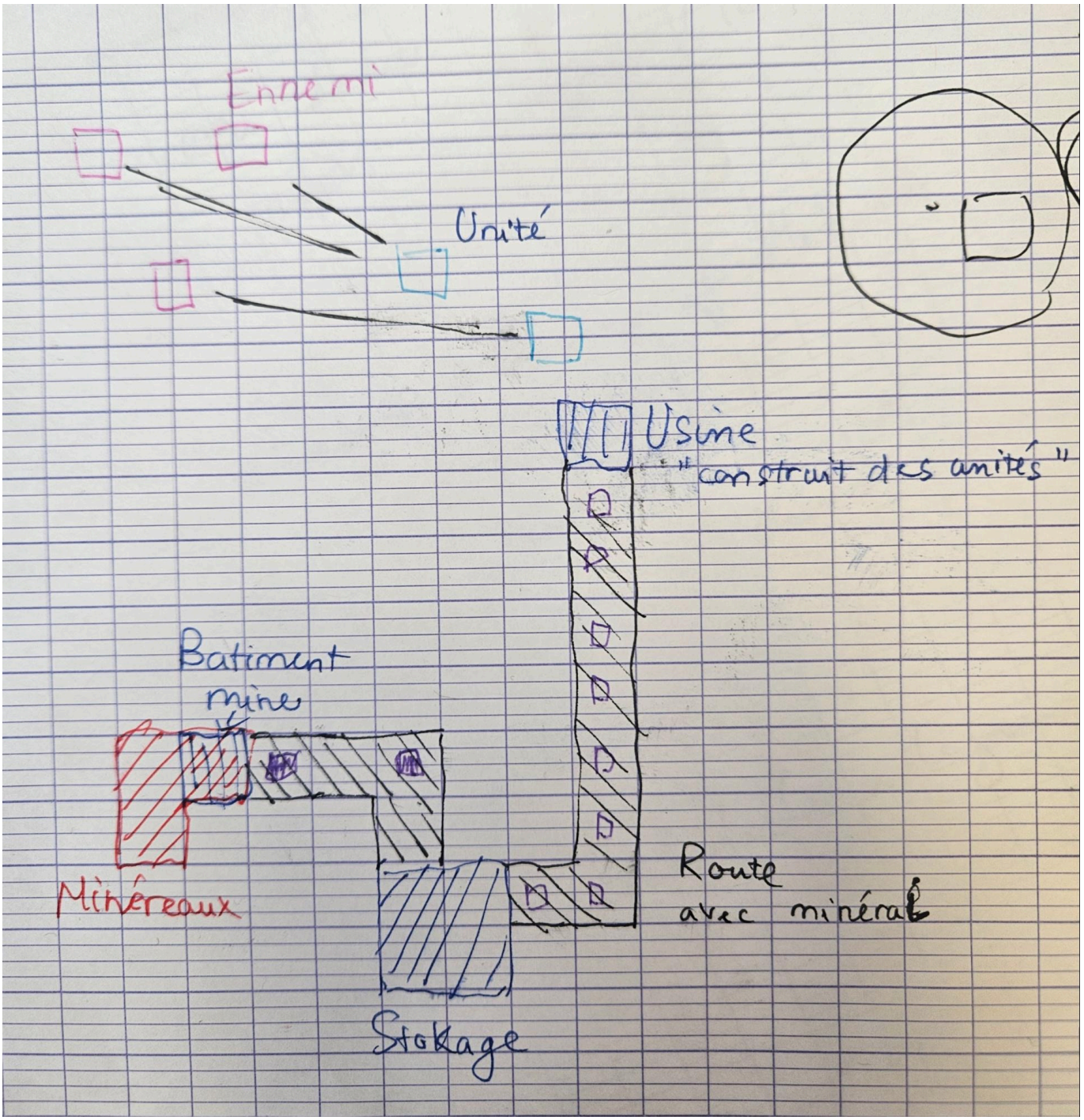
Visuellement, les bâtiments et minerais sont sur une grille. Les unités et les ennemis se déplacent de manière continue (ne suivent pas la grille).

Chaque case "route" possède une direction. Lorsqu'un minerai arrive sur sa case, elle le déplace dans cette direction uniquement si la case suivante est une route opérationnelle (pas cassée) qui ne contient pas de minerai, ou un bâtiment. Sinon, le minerai ne bouge pas. Si une route est détruite, seulement la case qui a été visée est détruite. Les minerais qui étaient en train d'être acheminés continuent leur chemin si possible. Le joueur décide lui-même la direction des routes pour avoir le contrôle complet sur l'acheminement des minerais, et pouvoir faire éventuellement des chemins plus complexes (ex: relier plusieurs chemins entre eux).

Prototypes visuels


(la zone de jeu est + grande que ça, c'est juste pour simplifier)
info sur l'élément sélectionné
unité
PV: 10
instantané
boutons pour changer d'action
entouré d'une couleur pour montrer que c'est sélectionné
(+ large)
Miner:
surligne les filons de minerai
cliquable
Sélectionnez un minerai
Défendre:
clique n'importe où tant que c'est vide
Instructions
visible
zone autour de l'unité


Route
état : vide
potentiellement menu pour changer de sens ?
Stockage
contient : 30
état : en marche
aucune action avec le stockage
Usine
machine
unité : 3
minerai manquant pour créer une unité supplémentaire
action des unités qui sortent ?
Vue d'ensemble
1
30
1
1
0
quand rien sélectionné (clique sur un espace vide)
info sur l'état du jeux



2. Analyse

Cette section présente les principales fonctionnalités à développer pour le projet ainsi que leur niveau de difficulté et leur priorité.

2.1 Analyse globale

Les principales fonctionnalités identifiées sont les suivantes :

- Gestion des coordonnées (grille discrète vs mouvement continu)
- Affichage des objets (unités, minerai, bâtiments)
- Gestion des actions des bâtiments (routes, usine, stockage, mine)
- Création, déplacements et actions des ennemis
- Génération du terrain en début de partie

- Actions et déplacements des unités
- Menus et interface utilisateur

Ces fonctionnalités seront détaillées dans la section suivante sous forme de sous-fonctionnalités, accompagnées d'une estimation du temps de développement, du niveau de difficulté et de la priorité. La priorité est évaluée sur une échelle de 1 à 5, où 1 correspond à une fonctionnalité essentielle à implémenter en premier et 5 à une fonctionnalité secondaire ou optionnelle.

2.2. Analyse détaillée

2.2.1. Gestion des coordonnées (grille discrète vs mouvement continu) - 1h45 Cette fonctionnalité constitue la base du système de déplacement et de collision entre les différents objets du jeu.

1. **Gestion des collisions entre ennemis/unités et bâtiments** — Difficulté : Moyenne — Priorité : 2 — Temps estimé : 1h
2. **Coordonnées sur la grille (bâtiments et minerais)** — Difficulté : Facile — Priorité : 1 — Temps estimé : 30 min
3. **Coordonnées des unités et ennemis (mouvement continu)** — Difficulté : Facile — Priorité : 1 — Temps estimé : 15 min

2.2.2. Affichage des objets (unités, minerai, bâtiments) - 2h30 L'affichage graphique permet une visualisation claire du terrain et des entités présentes.

1. **Fenêtre principale avec grille de jeu** — Difficulté : Facile — Priorité : 1 — Temps estimé : 45 min

2. **Affichage des unités** — Difficulté : Facile — Priorité : 2 — Temps estimé : 15 min

3. **Affichage des filons de minerai et des bâtiments** — Difficulté : Moyenne — Priorité : 2 — Temps estimé : 30 min

4. **Animation du minerai sur les routes (effet visuel)** — Difficulté : Moyenne — Priorité : 5 — Temps estimé : 1h

2.2.3. Gestion des bâtiments (usine, stockage, mine, routes) - 3h15 Les bâtiments forment l'infrastructure principale permettant l'extraction, le stockage, la transformation et l'acheminement des ressources.

1. **Usine** — Difficulté : Moyenne — Priorité : 3 — Temps estimé : 45 min

2. **Stockage des minerais** — Difficulté : Facile — Priorité : 2 — Temps estimé : 30 min

3. **Mine (stockage temporaire des minerais collectés)** — Difficulté : Moyenne — Priorité : 2 — Temps estimé : 45 min

4. **Déplacement des minerais via routes** — Difficulté : Moyenne — Priorité : 5 — Temps estimé : 1h15

2.2.4. Création, déplacements et actions des ennemis - 3h45 Les ennemis constituent un élément de pression continue qui oblige le joueur à organiser simultanément la défense et l'optimisation de la production.

1. **Génération automatique des ennemis par vagues** — Difficulté : Moyenne — Priorité : 4 — Temps estimé : 1h

2. **Déplacement des ennemis vers le bâtiment maître** — Difficulté : Moyenne — Priorité : 4 — Temps estimé : 45 min

3. **Actions des ennemis (destruction des bâtiments et unités à portée)** — Difficulté : Élevée — Priorité : 4 — Temps estimé : 1h30

4. **Mort des ennemis au contact des unités de défense** — Difficulté : Moyenne — Priorité : 4 — Temps estimé : 30 min

2.2.5. Génération du terrain en début de partie - 1h La génération du terrain établit l'état initial du jeu en positionnant les ressources, les bâtiments de départ et les unités sur la grille.

1. **Génération aléatoire des minerais** — Difficulté : Facile — Priorité : 2 — Temps estimé : 30 min

2. **Placement du bâtiment maître** — Difficulté : Facile — Priorité : 3 — Temps estimé : 15 min

3. **Création de l'unité de base initiale** — Difficulté : Facile — Priorité : 2 — Temps estimé : 15 min

2.2.6. Actions et déplacements des unités - 3h15 Les unités assurent l'exécution des ordres du joueur : récolte du minerai, construction des bâtiments et routes, transport des ressources et défense contre les ennemis.

1. **Miner** — Difficulté : Moyenne — Priorité : 2 — Temps estimé : 30 min

2. **Construction de bâtiments** — Difficulté : Moyenne — Priorité : 3 — Temps estimé : 30 min

3. **Transport du minerai** — Difficulté : Moyenne — Priorité : 2 — Temps estimé : 30 min

4. **Défense** — Difficulté : Élevée — Priorité : 3 — Temps estimé : 1h

5. **Attribution d'ordres aux unités** — Difficulté : Moyenne — Priorité : 2 — Temps estimé : 45 min

2.2.7. Menus et interface utilisateur – 3h15 (+30 min optionnel) L'interface utilisateur permet d'afficher les informations relatives aux ressources et aux bâtiments, ainsi que de commander les actions des unités de manière interactive.

1. **Gestion des clics de l'utilisateur** — Difficulté : Moyenne — Priorité : 1 — Temps estimé : 1h

2. **Menu pour les unités** — Difficulté : Moyenne — Priorité : 2 — Temps estimé : 1h30

3. **Menu des bâtiments (affichage des quantités de minerai)** — Difficulté : Moyenne — Priorité : 3 — Temps estimé : 30 min

4. **Vue d'ensemble des données (minerais, unités, bâtiments) (optionnel)** — Difficulté : Faible — Priorité : 5 — Temps estimé : 30 min

3. Plan de développement

Le projet est réalisé par un groupe de trois et représente un volume de travail total estimé à environ **75 heures**, incluant la planification, le développement et la rédaction du rapport.

Environ **50 heures** sont consacrées au développement des fonctionnalités du jeu, tandis qu'environ **15 heures** sont dédiées à la rédaction du rapport. Les **10 heures restantes** correspondent aux séances de

planification et à l'organisation du travail en équipe.

3.1 Tâches liées à la gestion du projet

- Conception & cahier des charges : 3 x 2h = 6 h
- Documentation & diagramme de Gantt : 10 h
- Utilisation et apprentissage de GitHub : 2 h

3.2 Répartition du temps par fonctionnalité

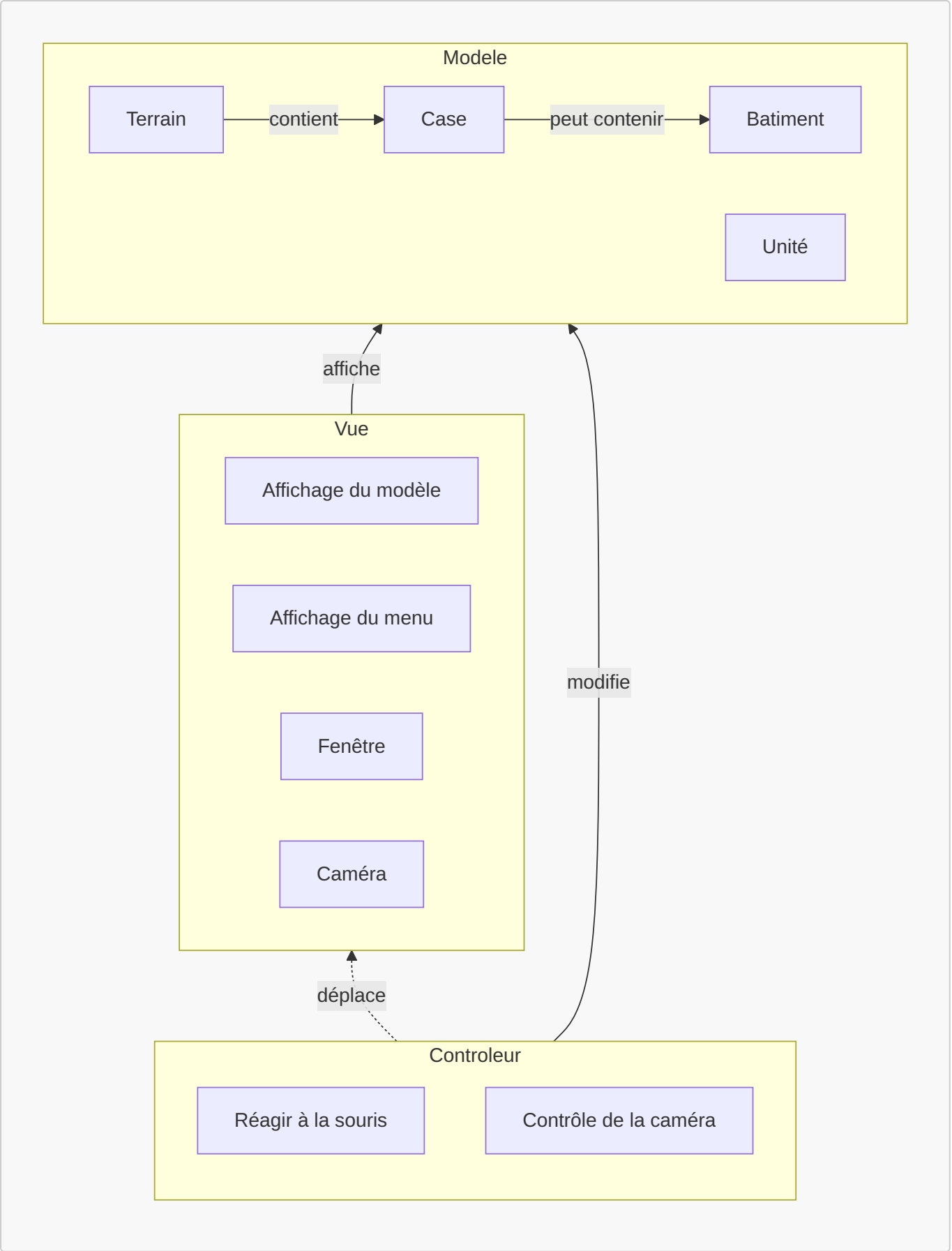
- Gestion des coordonnées (grille discrète vs mouvement continu): 1h45
- Affichage des objets (unités, minerai, bâtiments): 2h30
- Gestion des bâtiments (usine, stockage, mine, routes): 3h15
- Création, déplacements et actions des ennemis: 3h45
- Génération du terrain en début de partie: 1h
- Actions et déplacements des unités: 3h15
- Menus et interface utilisateur: 2h15 (+30 min optionnel)

3.3 Diagramme de Gantt

L'ensemble des tâches est présenté sous la forme d'un diagramme de Gantt, dont les ressources correspondent aux trois membres du groupe projet. Ce diagramme permet de visualiser la planification temporelle et le suivi de l'avancement du projet.

4. Conception générale

Le programme est basé sur une architecture MVC (Modèle Vue Contrôleur).



5. Conception détaillée

5.1 : Gestion des coordonnées (grille discrète vs mouvement continu)

5.1.1 : Gestion des collisions entre ennemis/unités et bâtiments

Structures de données et classes utilisées

- **Interface** `Selectable` : représente tout élément pouvant être ciblé ou détecté par le système de jeu, qu'il s'agisse d'une unité ou d'un bâtiment.
- **Classe** `CommandeDeplacementEnnemi` : gère le comportement de déplacement des ennemis. Elle scanne les cases autour de l'unité pour repérer une cible proche et ajuste la trajectoire en fonction de la distance d'attaque.
- **Classe** `CommandeDefendre` : gère le comportement défensif des ouvriers. Elle recherche les ennemis dans une zone de proximité, puis choisit entre déplacement et attaque selon la distance.
- **Classe** `Batiment` **/** `BatimentMaitre` : les bâtiments sont des cibles attaquables. Le bâtiment maître constitue une cible prioritaire dans le comportement des ennemis.

Principe de gestion

La version actuelle ne repose pas sur une collision physique au sens strict, mais sur une **gestion par proximité et par sélection de cible**. Les unités et les ennemis ne "rebondissent" pas sur les bâtiments ; à la place, ils détectent les éléments proches, choisissent une cible, puis s'arrêtent à une distance d'attaque adaptée avant de lancer une commande d'attaque.

Algorithme abstrait

1. **Détection de proximité** :
   - Les ennemis utilisent `CommandeDeplacementEnnemi.scanNearby(...)` pour inspecter les cases autour d'eux dans un rayon de `SCAN_RADIUS = 1`.
   - Les ouvriers en défense utilisent `CommandeDefendre.scanForEnemies(...)` pour repérer les ennemis proches dans une zone de deux cases environ.
2. **Sélection de la cible** :
   - Le système choisit la cible la plus proche parmi les éléments détectés.
   - Les ennemis ignorent les autres ennemis et ne ciblent que les éléments adverses détectables.
3. **Gestion de la distance d'attaque** :
   - Si la cible est trop loin, l'unité se déplace vers elle jusqu'à atteindre la portée d'attaque.
   - L'ennemi se positionne de manière à rester à `Ennemi.ATTACK_RANGE` de la cible avant d'attaquer.
   - L'ouvrier défensif applique la même logique avec `Ouvrier.DEFENSE_RANGE`.
4. **Attaque de la cible** :
   - Une fois la distance correcte atteinte, une commande d'attaque est ajoutée ou exécutée.
   - Dans le cas de la défense, la cible ennemie subit des dégâts périodiques après une phase de progression temporelle (`TEMPS`).

Conditions limites à respecter

- Il n'y a pas de véritable gestion de collision entre plusieurs unités mobiles. Elles peuvent donc se retrouver proches les unes des autres sans résolution de blocage.
- Le bâtiment maître est une cible particulière, traitée comme priorité dans la logique ennemie.

Interactions avec les autres fonctionnalités

- **Déplacement continu** : les coordonnées en pixels sont utilisées pour calculer les distances et adapter le déplacement des unités.

- **Affichage** : les positions des unités et bâtiments sont exploitées par le rendu pour afficher correctement les entités sur la grille.
- **Système de commandes** : la collision comportementale est intégrée au système de commandes séquentielles, ce qui permet aux unités de passer naturellement du déplacement à l'attaque.

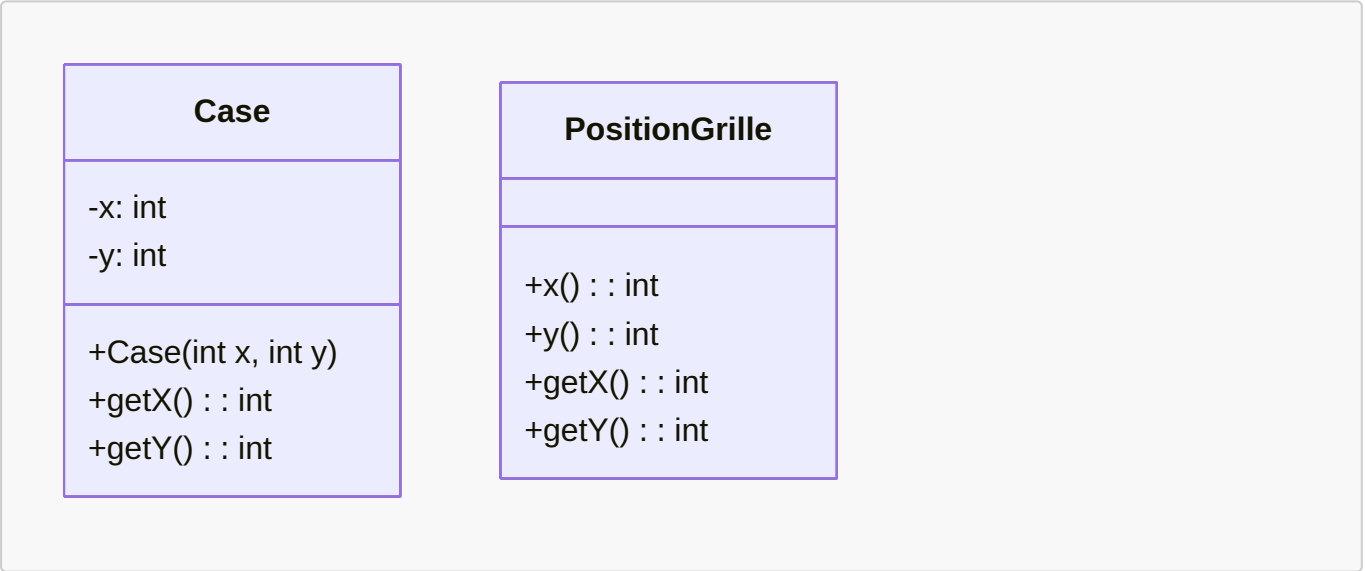
Remarque de conception

Nous avons décidé de mettre en œuvre cette stratégie simple pour les unités, car notre jeu peut comporter des centaines d'entités en mouvement qui modifient toutes simultanément l'état du jeu (`Terrain`). En effet, cela constituerait un véritable cauchemar en termes de performances si chacune d'entre elles devait constamment vérifier les collisions et exécuter des algorithmes de calcul d'itinéraire.

5.1.2 : Coordonnées sur la grille (bâtiments et minerais)

La classe `Case` représente une case de la grille de jeu. Elle contient des coordonnées discrètes (x, y) qui correspondent à sa position sur la grille. Les bâtiments et les minerais sont placés sur ces cases, et leurs coordonnées sont donc alignées avec la grille.

Pour manipuler plus facilement ces coordonnées, on utilise `PositionGrille` sous forme de *record* immuable, ce qui garantit notamment une comparaison correcte par valeur (`equals`/`hashCode`) pour les collections.



5.1.3 : Coordonnées des unités et ennemis (mouvement continu)

Structures de données utilisées

- **Classe `Unite`** : Base pour toutes les unités (ouvriers, ennemis), stocke les coordonnées continues en pixels (`px`, `py`) et la vitesse de déplacement (`speed`).
- **Classe `Ennemi`** : Étend `Unite` avec des constantes spécifiques (HP_INITIAL, VITESSE, DEGATS, ATTACK_RANGE).
- **Classe `CommandeDeplacement`** : Gère le déplacement vers une destination en pixels. Calcule le vecteur de direction et applique un pas de déplacement proportionnel à la vitesse et au temps écoulé.
- **Classe `CommandeDeplacementEnnemi`** : Spécialisation pour les ennemis, implémente un scan de la zone régulière (SCAN_RADIUS) pour détecter des cibles et ajuster la trajectoire.

Constantes du modèle

- `Unite,VITESSE` : 0.2f pour les ouvriers, 0.15f pour les ennemis
- `Case.TAILLE` : Taille d'une case en pixels, utilisée pour la conversion grille ↔ pixels via `getGX()`, `getGY()`
- `Ennemi.ATTACK_RANGE` : Case.TAILLE (portée d'attaque)
- `CommandeDeplacementEnnemi.SCAN_RADIUS` : 1 (rayon de détection de 2 cases)

Algorithme abstrait

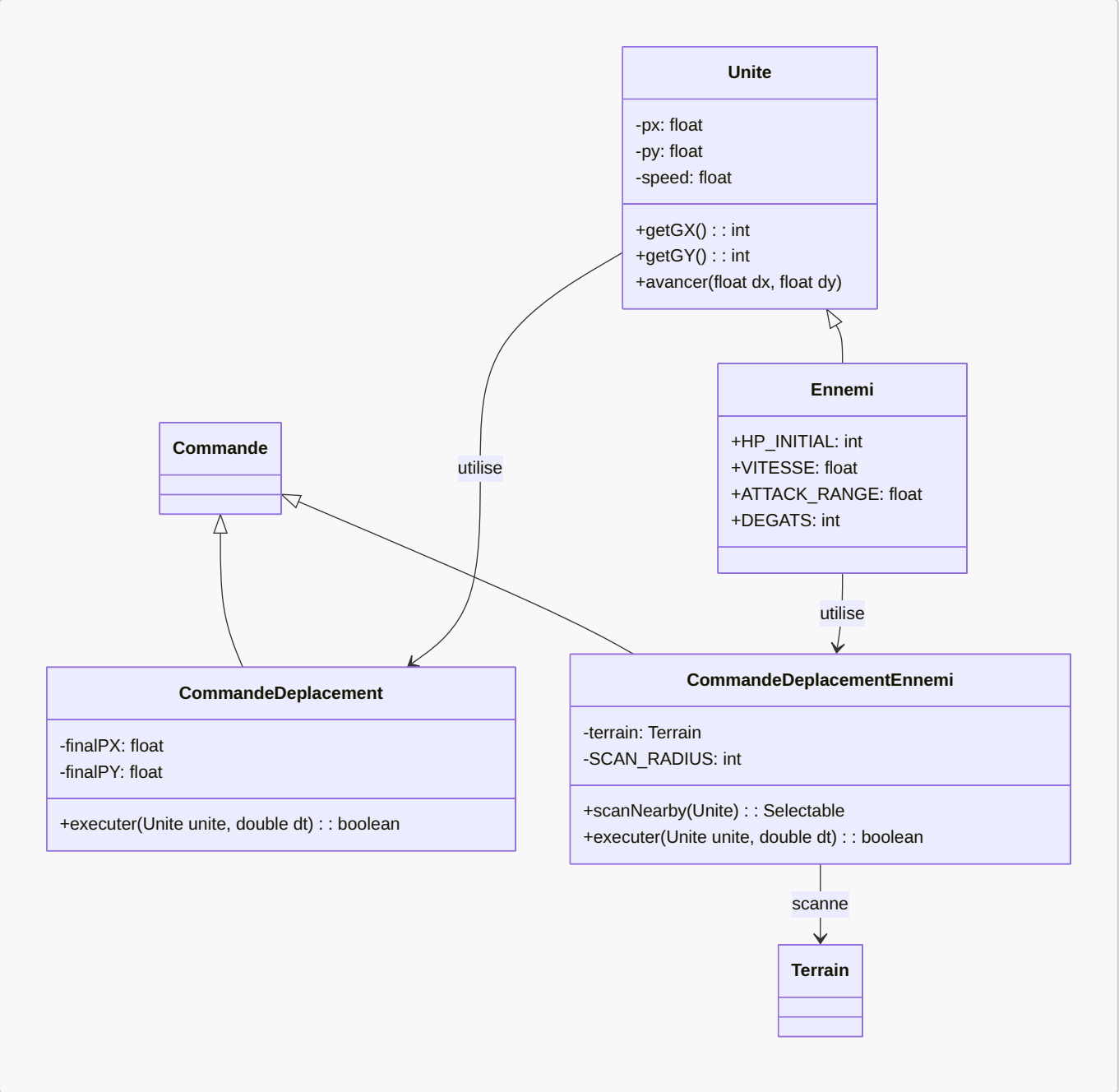
1. **Initialisation** : Une unité est créée avec des coordonnées initiales en pixels (`px = GX * Case.TAILLE`, `py = GY * Case.TAILLE`).
2. **Conversion des coordonnées** : `getGX()` et `getGY()` convertissent les pixels en coordonnées de grille par division par `Case.TAILLE`.
3. **Déplacement** : `CommandeDeplacement.executer()` calcule la distance vers la destination et applique un pas (`step = speed * dt`). Si la distance est inférieure au pas, l'unité atteint directement la destination. Sinon, elle avance proportionnellement.
4. **Détection et poursuite (ennemis)** : `CommandeDeplacementEnnemi` scanne un rayon de 2 cases autour de l'ennemi. Si une cible (bâtiment ou unité allié) est détectée, l'ennemi ajuste sa trajectoire pour maintenir une distance d'attaque optimale, puis déclenche une `CommandeAttaquer`.

Conditions limites à respecter

- La conversion pixels ↔ grille doit être cohérente pour éviter les erreurs de positionnement.
- Les déplacements doivent respecter les limites du terrain.
- Les ennemis ne doivent pas se bloquer mutuellement (pas de gestion de collision entre ennemis dans la version actuelle).

Interactions avec les autres fonctionnalités

- **Terrain** : Fournit les dimensions et les obstacles pour le calcul des trajectories.
- **Système de commandes** : Les déplacements sont gérés via une file de commandes séquentielles (Deque).
- **Affichage** : Les coordonnées pixels sont directement utilisées par `AffichageUnites` pour dessiner les unités.



5.2 : Affichage des objets (unités, minerai, bâtiments)

5.2.1 : Fenêtre principale avec grille de jeu

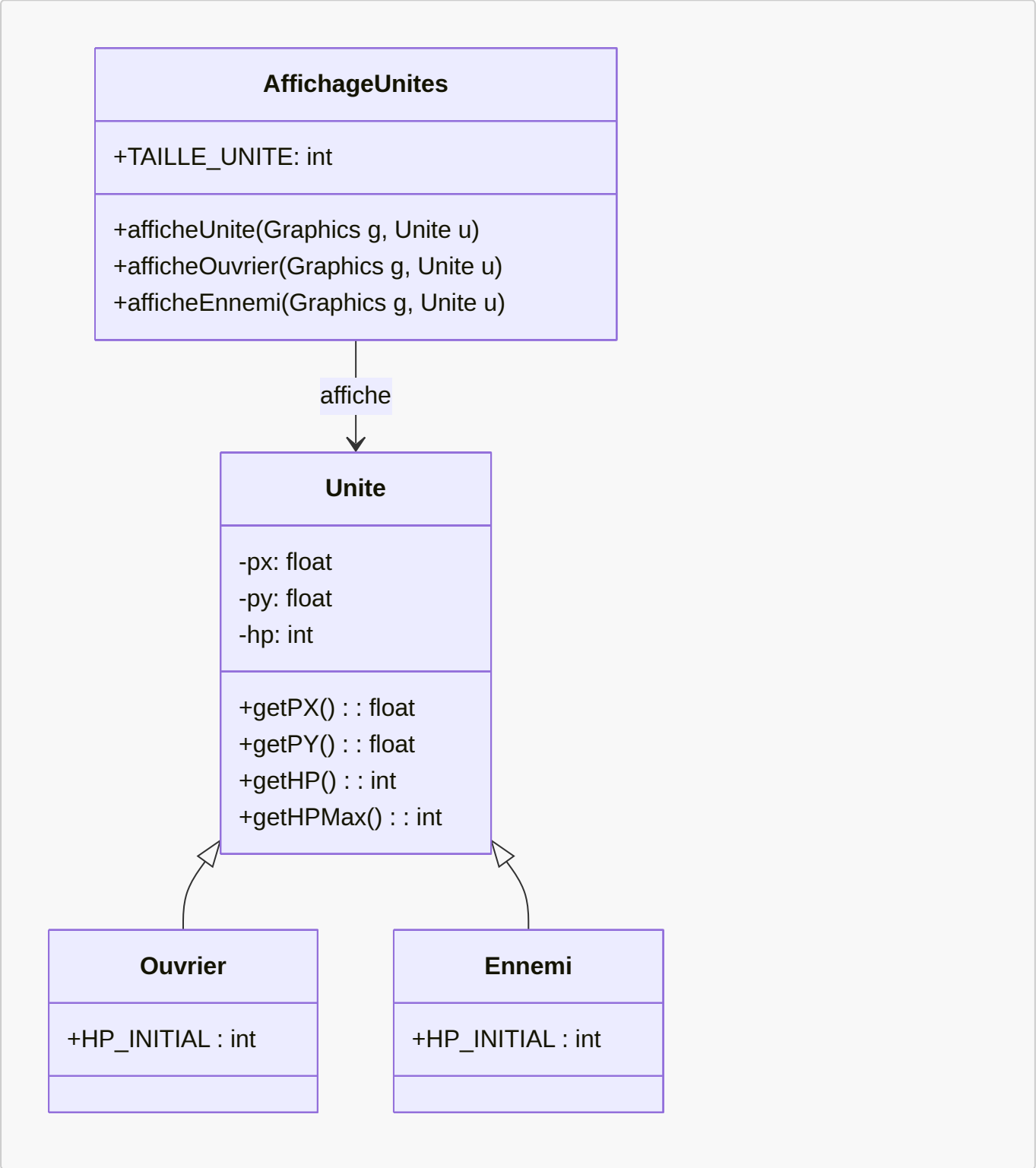
- Classe `Fenetre` : classe principale d'affichage, hérite de `JFrame`
  - Constructeur `Fenetre(String titre, Terrain terrain)` : initialise la fenêtre, ajoute les composants graphiques nécessaires (comme l'affichage du terrain) et rend la fenêtre visible.
- Classe `Affichage` : responsable de l'affichage du terrain de jeu, hérite de `JPanel`
  - Attribut : `terrain` (référence au terrain à afficher)
  - Méthode `paintComponent(Graphics g)` : surcharge pour dessiner les éléments du terrain (minerais, bâtiments, etc.) en fonction de leur position et de leur type. Appelle les fonctions d'affichage spécifiques dans `AffichageMinerais` et `AffichageBâtiments`. Garde un espace pour afficher le menu à droite de la grille.

5.2.2 : Affichage des unités

La classe `AffichageUnites` gère l'affichage des unités (ennemis et ouvriers) sur la grille. Chaque unité est dessinée sous forme de cercle centré sur ses coordonnées en pixels.

Les `Ouvriers` sont des cercles initialement magenta, et deviennent de plus en plus violets à mesure qu'ils perdent des PV. Les `Ennemis` sont représentés sous la forme de triangles rouges.

La fonction `afficheUnite(Graphics g, Unite u)` affiche une unité en fonction de son type (ouvrier ou ennemi) et de ses points de vie. Elle utilise les méthodes `afficheOuvrier` et `afficheEnnemi` pour différencier les types d'unités.



5.2.3 : Affichage des filons de minerai et des bâtiments

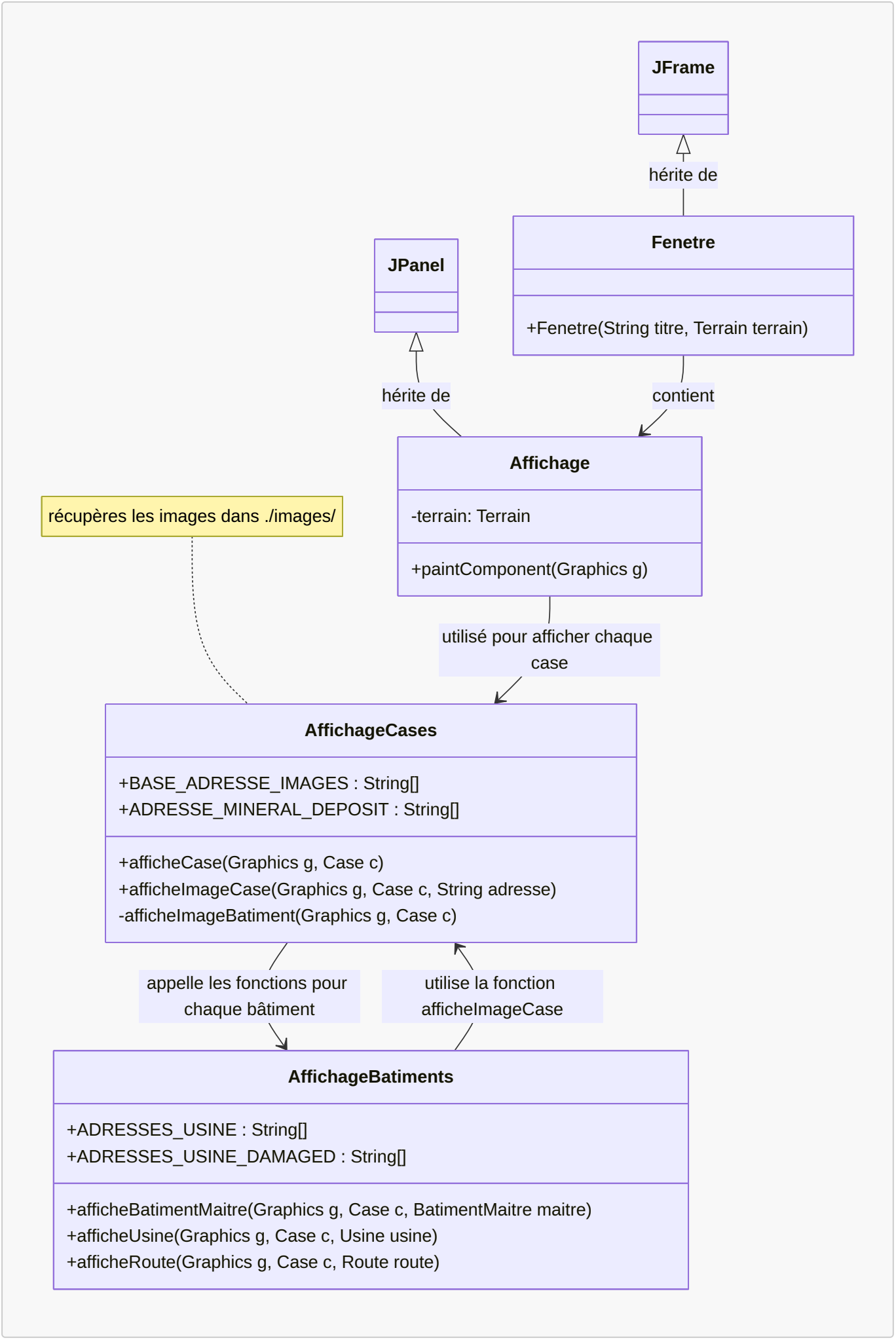
La classe `AffichageCases` contient des méthodes statiques pour afficher les différents types de cases (vide, minerai, bâtiment) à une position donnée sur la fenêtre. Les images utilisées sont chargées depuis le dossier `./images/`.

- Méthodes statiques :
  - `afficheCase(Graphics g, Case c)` : affiche une case en fonction de son type (vide, minerai, bâtiment) en appelant la méthode d'affichage correspondante.
  - `afficheImageCase(Graphics g, Case c, String adresse)` : affiche l'image à l'emplacement `adresse` sur la case `c`. Récupère l'image dans le cache si on l'a déjà chargée.
  - `afficheImageBatiment(Graphics g, Case c)` : affiche une image correspondant au bâtiment contenu dans la case `c` en appelant la méthode correspondante dans `AffichageBatiments`.
- Variables statiques :
  - `BASE_ADRESSE_IMAGES` : l'adresse correspondant au dossier `images/` pour l'ordinateur de l'utilisateur.
  - `ADRESSE_MINERAL_DEPOSIT`, `ADRESSE_EN_CONSTRUCTION` : les adresses des sprites utilisés pour l'affichage.

La classe `AffichageBatiments` contient les méthodes et les adresses d'images spécifiques à l'affichage des bâtiments. En plus du type de batiment, l'affichage dépend de la quantité de PV que le bâtiment a, son état de fonctionnement (usine et foreuse), et la quantité de minerai qu'il contient (stockage et batiment maitre).

- Méthodes statiques :
  - `affiche{nom du batiment}(Graphics g, Case c, {type du batiment} b)` : affiche une image du batiment en question à la position de la case sur la fenêtre.
- Variables statiques :
  - `ADRESSES_{nom du batiment}` : tableau d'adresses des sprites du batiment en question en fonction de son état, et un sprite du bâtiment détruit.
  - `ADRESSES_{nom du batiment}_DAMAGED` : tableau d'adresses des sprites de dégâts du bâtiment en question, à superposer à l'image de base en fonction de la quantité de PV restante du bâtiment.

Les adresse des images utilisées sont stockées dans différentes variables statiques, dans `AffichageBatiments` pour celles spécifiques à chaque batiments et dans `AffichageCases` pour les autres. On utilise un HashMap comme cache afin d'éviter de recharger la même image plusieurs fois.


JFrame
hérite de
Fenetre
+Fenetre(String titre, Terrain terrain)
JPanel
hérite de
contient
Affichage
-terrain: Terrain
+paintComponent(Graphics g)
récupères les images dans ./images/
utilisé pour afficher chaque case
AffichageCases
+BASE_ADRESSE_IMAGES : String[]
+ADRESSE_MINERAL_DEPOSIT : String[]
+afficheCase(Graphics g, Case c)
+afficheImageCase(Graphics g, Case c, String adresse)
-afficheImageBatiment(Graphics g, Case c)
appelle les fonctions pour chaque bâtiment
utilise la fonction afficheImageCase
AffichageBatiments
+ADRESSES_USINE : String[]
+ADRESSES_USINE_DAMAGED : String[]
+afficheBatimentMaitre(Graphics g, Case c, BatimentMaitre maitre)
+afficheUsine(Graphics g, Case c, Usine usine)
+afficheRoute(Graphics g, Case c, Route route)

5.2.4 : Animation du minerai sur les routes (effet visuel)

Les minerais en transit sont affichés de manière continue avec la classe `AffichageMinerais`. Chaque minerai utilise une image (`sprite_crystal_centre.png`) qui se déplace fluidement entre les cases de la grille. L'animation utilise des coordonnées en pixels (px, py) pour un rendu visuel fluide, avec interpolation linéaire entre la position de départ et la position cible à chaque étape de progression.

Caractéristiques :

- Image : `sprite_crystal_centre.png` (taille réduite à la moitié d'une case)
- Pas de barre de progression jaune (supprimée pour un rendu plus épuré)
- Le minerai disparaît automatiquement lorsqu'il atteint une destination finale (bâtiment maître ou stockage)

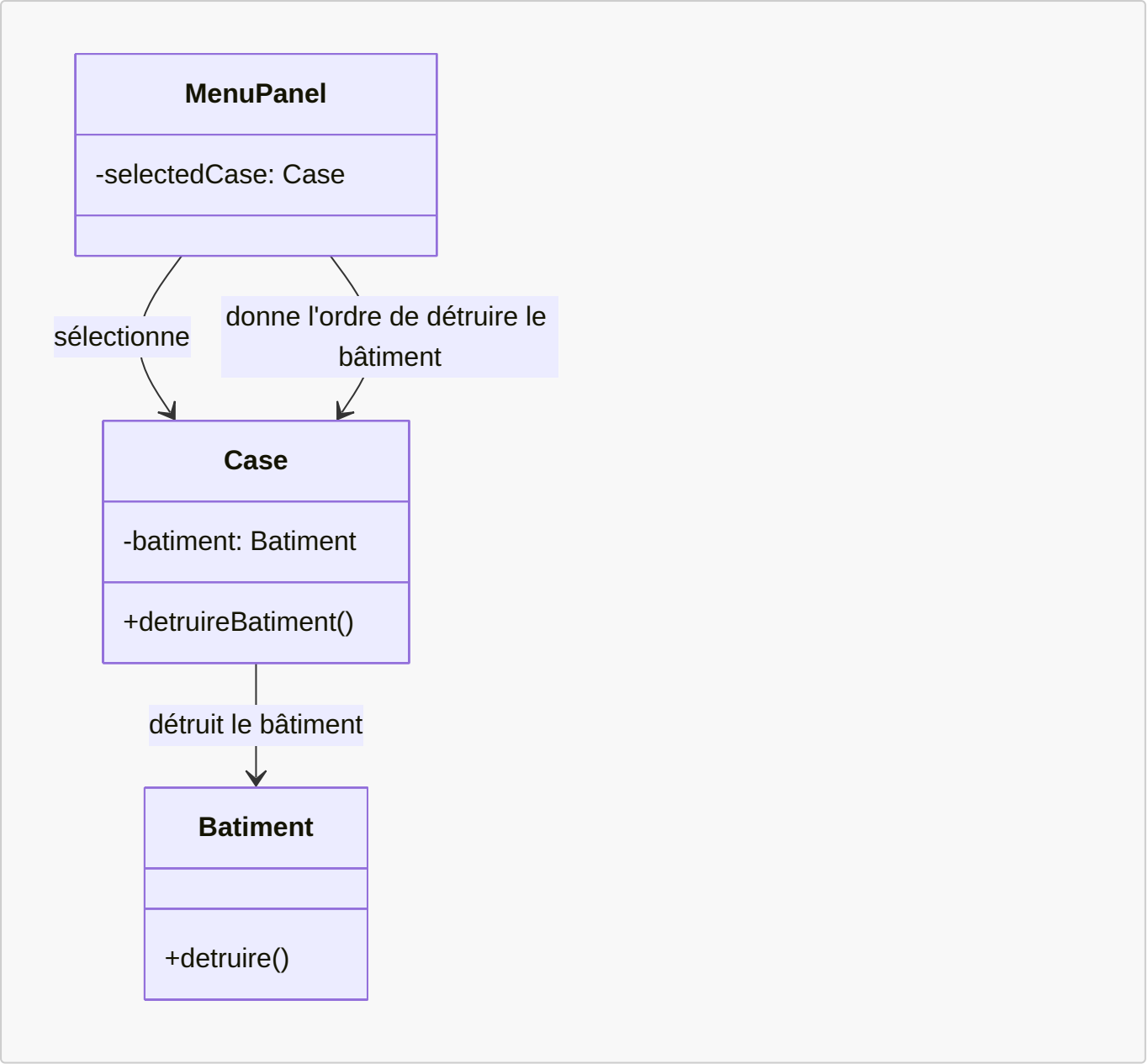
La méthode `afficheMinerai(Graphics g, Minerai minerai)` gère l'affichage de chaque minerai en fonction de sa position actuelle et de son état de progression.

5.3 : Gestion des bâtiments (usine, stockage, mine, routes)

5.3.0 : Destruction des bâtiments

Lorsque l'utilisateur clique sur une case contenant un bâtiment (sauf le bâtiment maître), un bouton "Détruire" apparaît dans le menu d'actions. En cliquant dessus, le bâtiment est retiré de la case correspondante, et l'affichage est mis à jour pour refléter la suppression du bâtiment.

Chaque bâtiment contient une méthode `detruire()` qui gère la logique de destruction (ex: libérer les ressources, mettre à jour l'état de la case, etc.). Le `MenuPanel` appelle la méthode `detruireBatiment()` de la `Case`, qui appelle la méthode `detruire()` sur le bâtiment et met la variable `batiment` à `null`.



5.3.5 : Fin de partie (Game Over)

Le jeu se termine automatiquement lorsque le bâtiment maître est détruit (HP atteignant 0). Un écran de Game Over s'affiche alors par-dessus l'interface du jeu sans Fermer l'application.

Structures de données et constantes

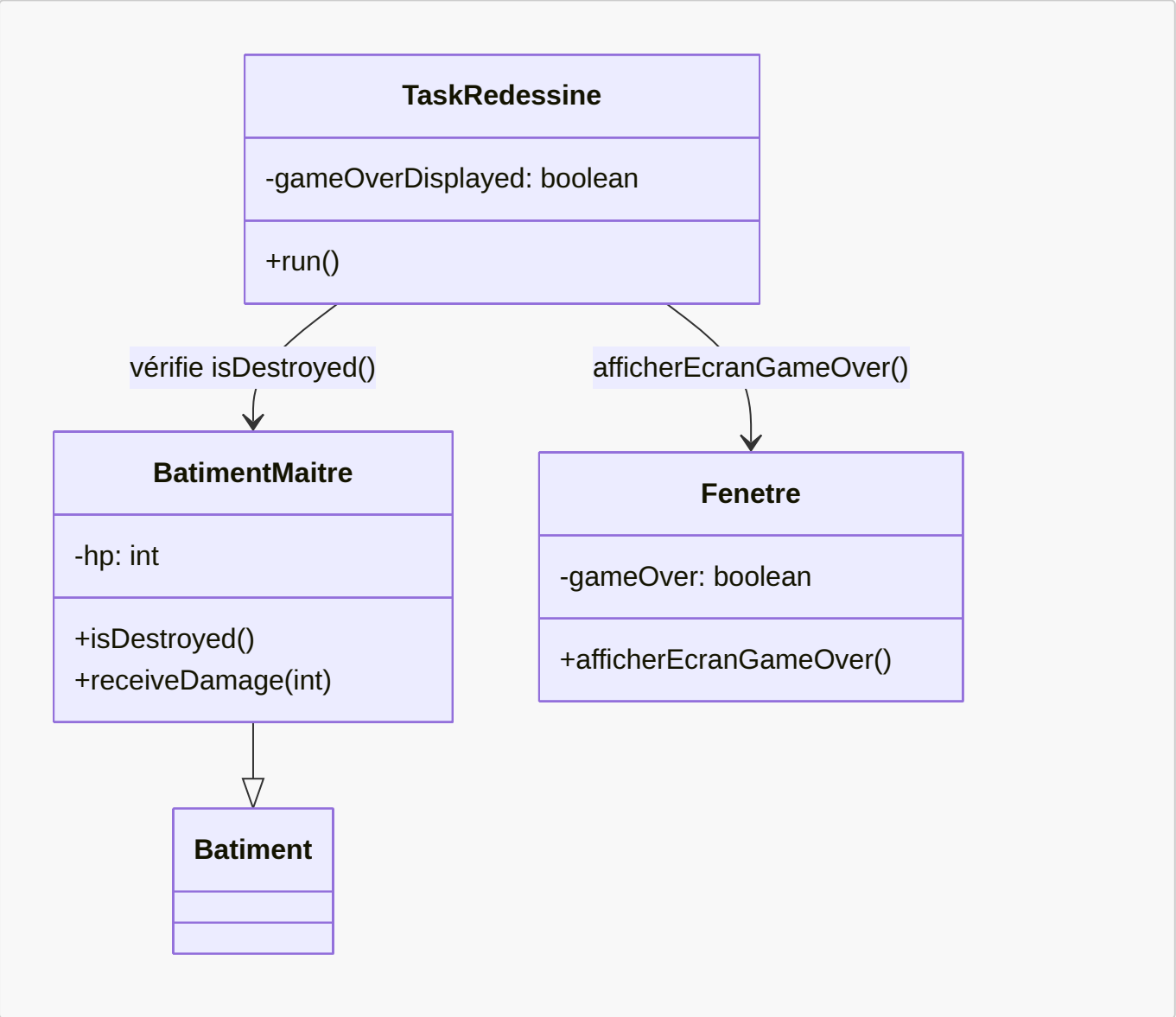
- **Classe `BatimentMaitre`** : Gère les points de vie du bâtiment principal
  - `HP_MAX`: points de vie maximum (100 par défaut)
  - `isDestroyed()`: méthode héritée de `Batiment` pour vérifier si HP ≤ 0
- **Classe `Fenetre`** : Gère l'affichage de l'écran de fin de partie
  - `gameOver`: booléen indiquant si le jeu est terminé
  - `afficherEcranGameOver()`: affiche un écran semi-transparent rouge avec le message "GAME OVER"
- **Classe `TaskRedessine`** : Détecte la fin de partie et déclenche l'affichage
  - Vérifie à chaque tick si `batimentMaitre.isDestroyed()`
  - Utilise le GlassPane de JFrame pour superposer l'écran de Game Over

Algorithme

1. À chaque cycle de rendu (16ms), `TaskRedessine.run()` vérifie l'état du bâtiment maître
2. Si le bâtiment maître est détruit (`isDestroyed() == true`) et que le Game Over n'a pas encore été affiché :
   - Appelle `Fenetre.afficherEcranGameOver()`
   - Marque `gameOverDisplayed = true` pour éviter les affichages multiples
3. L'écran de Game Over utilise le **GlassPane** de Swing pour superposer un panneau semi-transparent rouge par-dessus tout le jeu
4. Le jeu continue de s'exécuter en arrière-plan (les threads ne sont pas arrêtés)

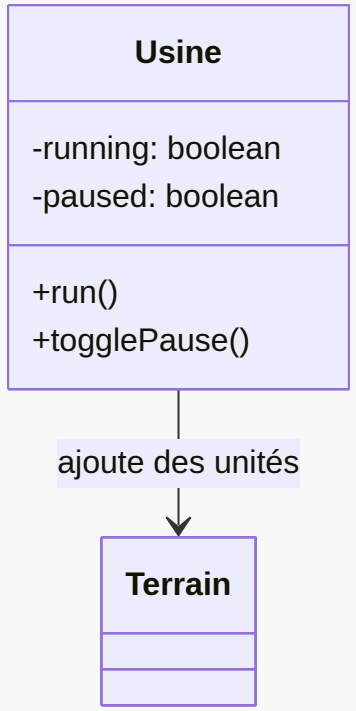
Utilisation

- La destruction du bâtiment maître peut être déclenchée par :
  - `batimentMaitre.receiveDamage(X)`: inflige X dégâts au bâtiment
  - `batimentMaitre.detruire()`: détruit immédiatement le bâtiment (met HP à 0)
- L'écran de Game Over est purement visuel et n'affecte pas la logique du jeu



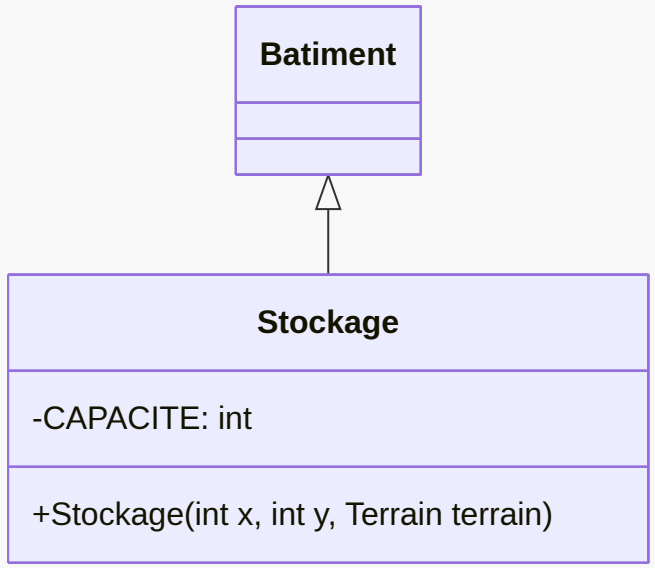
5.3.1 : Usine

La classe `Usine` produit des unités à partir de minerai. Elle fonctionne en arrière-plan via un thread qui consomme des minerais et crée des unités à intervalles réguliers. L'usine peut être mise en pause ou arrêtée.



5.3.2 : Stockage des minerais

La classe `Stockage` permet de stocker des minerais avec une capacité fixe. Elle hérite de `Batiment` et gère les opérations d'ajout et de retrait de minerais.



5.3.3 : Mine (extraction automatique de minerai via des threads)

Structures de données principales et constantes

- Classe `Foreuse` (hérite de `Batiment`, implémente `Runnable`) : gère l'extraction automatique du minerai.
- Classe `Minerai` (implémente `Runnable`) : transporte une unité de minerai le long des routes.
- Classe `AsyncExecutor` : centralise l'exécution asynchrone via un `ExecutorService`.
- Attributs :
  - `DELAI_EXTRACTION` (constante, délai entre deux extractions)
  - `running` (flag d'arrêt du thread)
  - stockage (hérité de `Batiment`, capacité 1)
- Méthodes :
  - `run()` (boucle d'extraction)
  - `arreter()` (arrêt du thread)
  - `ajouterMinerai(int)` (hérité)
    - `AsyncExecutor.runAsync(...)` (lancement centralisé des tâches)

Algorithme abstrait

1. Lorsqu'une foreuse est placée sur une case MINERAI, une tâche asynchrone est lancée (méthode `run`).
2. Tant que le flag `running` est vrai, la foreuse attend `DELAI_EXTRACTION` millisecondes.
3. Si le stockage n'est pas plein, elle ajoute 1 minerai à son stockage (méthode héritée de `Batiment`).
4. À chaque extraction, une tâche `Minerai` est lancée via `AsyncExecutor` pour le transport.
5. Si le stockage est plein, elle attend le prochain cycle.
6. Le thread peut être arrêté proprement via la méthode `arreter()` (flag `running` mis à false).

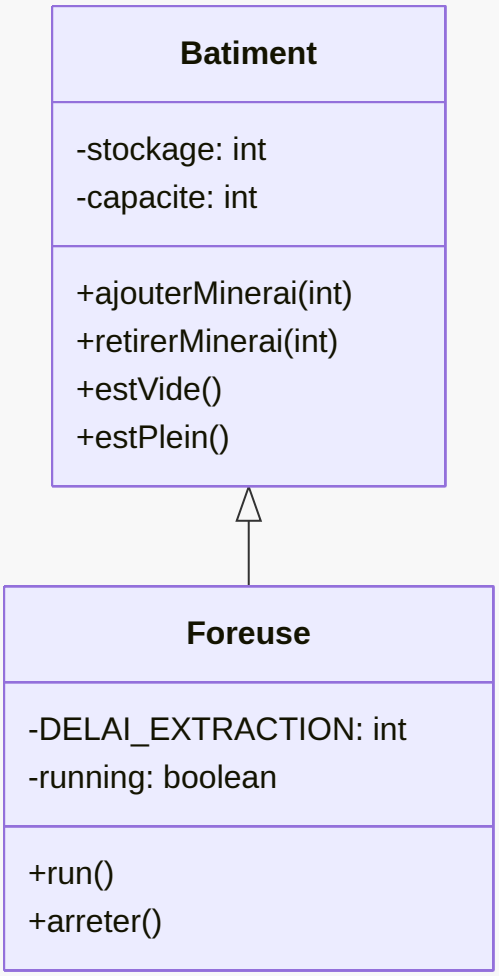
Conditions limites à respecter

- Le stockage ne doit jamais dépasser 1 (capacité fixée à la création).
- La foreuse ne doit extraire que si la case contient du minerai.
- Le thread doit pouvoir être arrêté proprement (interruption, flag `running` volatile).
- Les accès concurrents au stockage doivent être sûrs (pas de bug de synchronisation, ici géré par la simplicité du modèle).

Utilisation par les autres fonctionnalités

- Les routes peuvent venir vider le stockage de la foreuse pour transporter le minerai.
- L'interface peut afficher l'état du stockage en temps réel.
- Les tests de threads (`ForeuseThreadTest`, `MineraiTest`) vérifient le comportement asynchrone et la synchronisation.

Diagramme de classe simplifié



Côté Minerai

La classe `Minerai` modélise le transport d'une unité de ressource sur la carte.

- `Minerai` implémente `Runnable`.
- La position courante est stockée via `x` et `y`.
- Le déplacement s'exécute par pas temporels (`DELAI_TRANSPORT`).
- Le drapeau `running` est `volatile` pour permettre un arrêt propre depuis un autre thread.

Le transport est lancé de manière asynchrone par la foreuse via l'exécuteur centralisé, puis le minerai avance case par case jusqu'à atteindre une route valide, le bâtiment maître, ou une condition d'arrêt.

5.3.4 : Déplacement des minerais via routes

Structures de données principales

- `Minerai` : unité mobile de transport sur la grille.
- `Route` : bâtiment intermédiaire avec direction (`NORD`, `SUD`, `EST`, `OUEST`).
- `BatimentMaitre` : destination finale du minerai.
- `Case` : accès à l'état local (présence d'un bâtiment, type de case).

Algorithme abstrait

1. Le minerai démarre sur la position de la foreuse.
2. À chaque cycle, le minerai lit la case courante.
3. Le minerai vérifie d'abord que le bâtiment courant est terminé (`estFini()`). Si ce n'est pas le cas, il attend le cycle suivant.
4. Si la case courante est une route, il calcule la prochaine case selon la direction de la route.
5. Sinon, il recherche la meilleure direction disponible (route/destination finale valide).
6. Si la prochaine case est hors limites, le minerai attend et retente au cycle suivant.
7. La case cible doit contenir un bâtiment terminé (`estFini()`) et avec de la capacité disponible.
8. Si les conditions source/cible sont valides, le minerai est transféré (ajout en cible puis retrait en source, avec rollback en cas d'échec).
9. Si la cible est le bâtiment maître ou un stockage, le transport se termine pour ce minerai.

Conditions limites à respecter

- Une route ne peut contenir qu'un seul minerai à la fois.
- Un minerai ne peut se déplacer que depuis/vers des bâtiments dont la construction est terminée (`estFini()`).
- Le retrait du minerai depuis la case précédente doit rester cohérent avec l'ajout sur la case suivante.
- Le déplacement ne doit jamais sortir des bornes de la grille.
- L'arrêt doit être sûr même en contexte concurrent (flag `running` + interruption).
- **Le minerai disparaît visuellement lorsqu'il atteint le bâtiment maître OU un stockage** (modification récente).

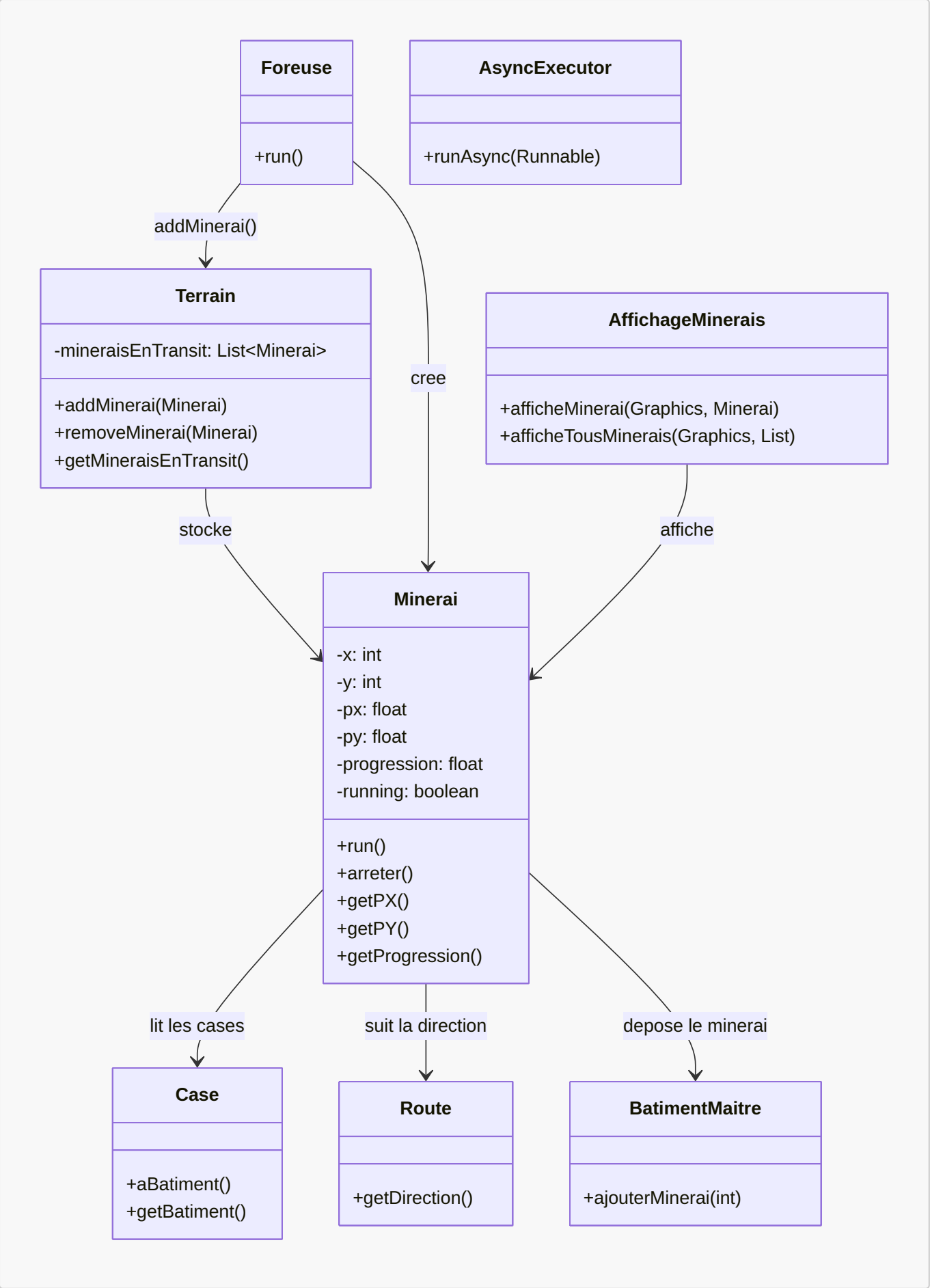
Complexité

Pour un minerai donné, la complexité temporelle est linéaire par rapport au nombre de cases parcourues jusqu'à la destination ou l'arrêt.

Utilisation par les autres fonctionnalités

- Alimente le bâtiment maître en ressources.
- Reflète l'état logistique du réseau de routes.
- Sert de base pour les futures animations visuelles du transport dans la vue.

Diagramme de classe simplifié



Implémentation améliorée :

- Les minerais ont maintenant des coordonnées en **pixels** (`px`, `py`) pour un déplacement fluide

- Une **animation de déplacement** avec interpolation linéaire entre les cases
- Un **indicateur de progression circulaire** s'affiche autour des minerais en mouvement
- La liste `mineraisEnTransit` dans `Terrain` permet à `AffichageTerrain` de tous les afficher

5.4 : Création, déplacements et actions des ennemis

5.4.0 : Gestion des vagues d'ennemis

Le jeu implémente un système de **vagues d'ennemis** qui attaquent le bâtiment maître à intervalles réguliers.

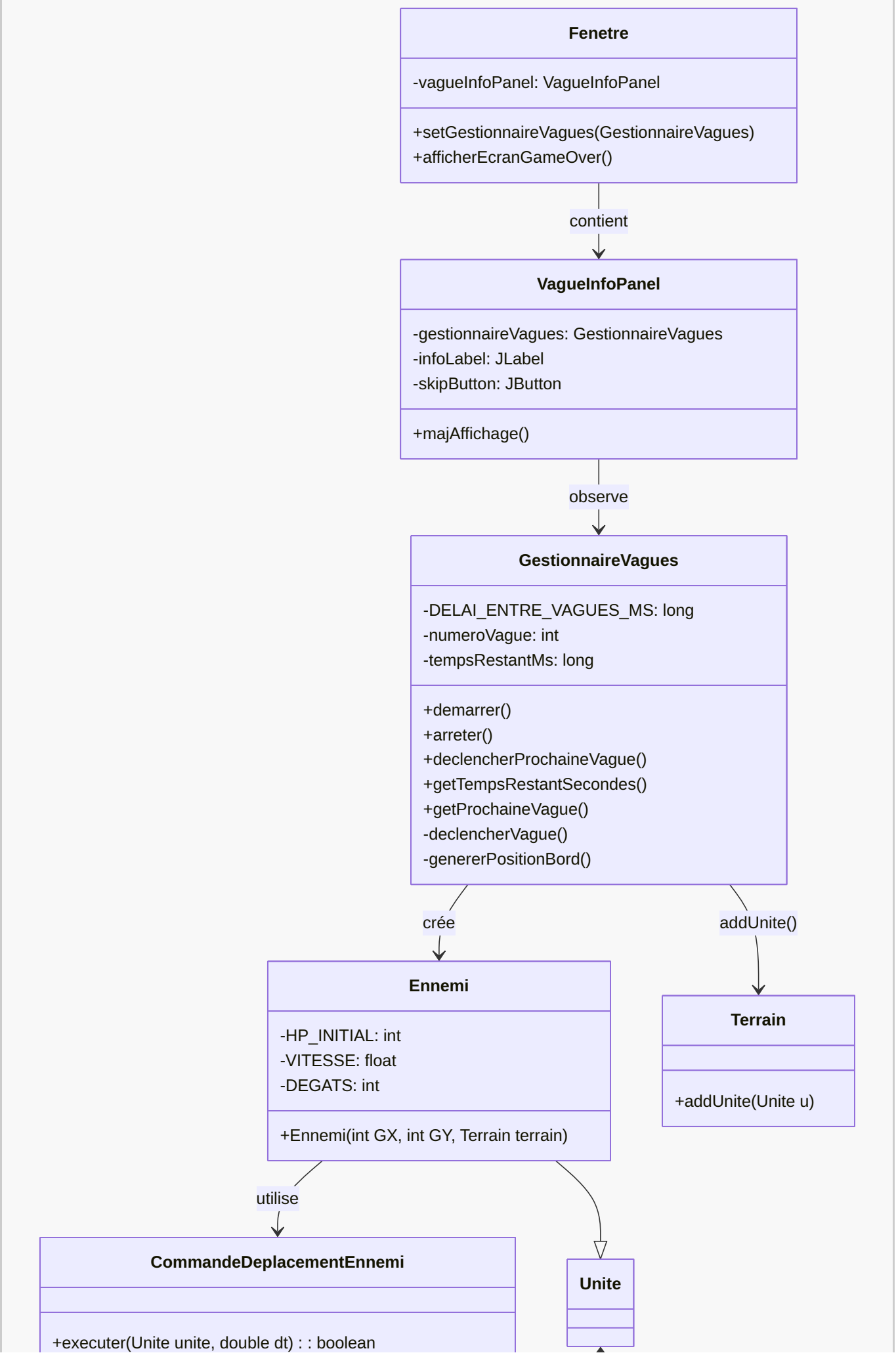
Structures de données et constantes

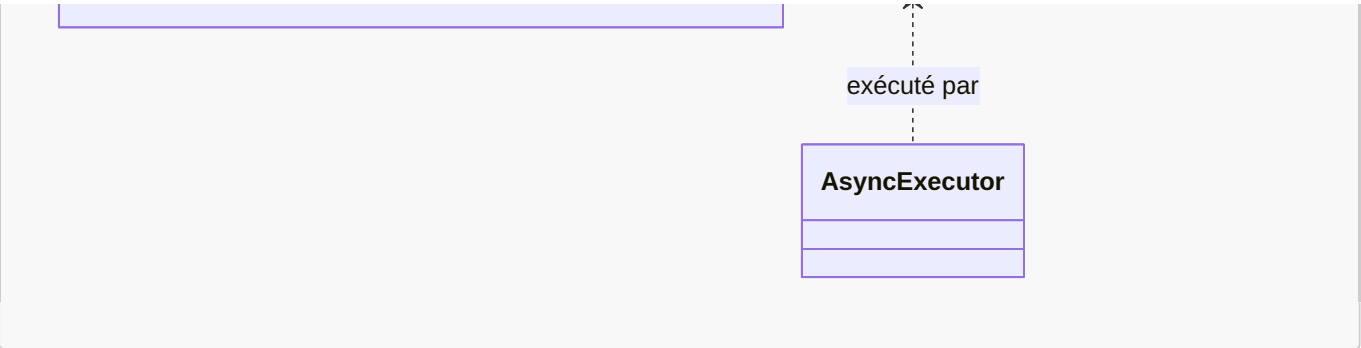
- **Classe `GestionnaireVagues`** : Gère la génération périodique des vagues d'ennemis
  - `DELAI_ENTRE_VAGUES_MS`: 3 minutes (180 000 ms) entre chaque vague
  - `numeroVague`: compte le nombre de vagues écoulées
  - `tempsRestantMs`: temps restant avant la prochaine vague
- **Classe `Ennemi`** : Représente un ennemi avec :
  - `HP_INITIAL`: 8 points de vie
  - `VITESSE`: 0.25 (vitesse de déplacement)
  - `DEGATS`: 2 (dégâts infligés par attaque)
  - `ATTACK_RANGE`: taille d'une case (portée d'attaque)
- **Classe `VagueInfoPanel`** : Panneau d'interface affichant :
  - Le numéro de la prochaine vague
  - Le compte à rebours (MM:SS)
  - Un bouton "Sauter le temps" pour déclencher manuellement la prochaine vague

Algorithme

1. **Initialisation** : Le `GestionnaireVagues` est créée et démarrée dans `Main`
2. **Déclenchement des vagues** : Toutes les 3 minutes, une nouvelle vague est déclenchée
3. **Nombre d'ennemis par vague** : La vague n°N contient N ennemis (vague 1 = 1 ennemi, vague 2 = 2 ennemis, etc.)
4. **Positionnement** : Les ennemis apparaissent aléatoirement sur les bords de la carte (haut, bas, gauche, droite)
5. **Comportement** : Chaque ennemi se déplace vers le bâtiment maître via `CommandeDeplacementEnnemi`
6. **Gestion du cycle de vie** : Les threads des ennemis sont gérés par `AsyncExecutor` via `Terrain.addUnite()`
7. **Affichage du compteur** : Un timer séparé met à jour le temps restant chaque seconde, affiché dans `VagueInfoPanel`
8. **Bouton de saut** : L'utilisateur peut cliquer sur "Sauter le temps" pour déclencher immédiatement la prochaine vague

Diagramme de classes


Fenetre
-vagueInfoPanel: VagueInfoPanel
+setGestionnaireVagues(GestionnaireVagues)
+afficherEcranGameOver()
contient
VagueInfoPanel
-gestionnaireVagues: GestionnaireVagues
-infoLabel: JLabel
-skipButton: JButton
+majAffichage()
observe
GestionnaireVagues
-DELAI_ENTRE_VAGUES_MS: long
-numeroVague: int
-tempsRestantMs: long
+demarrer()
+arreter()
+declencherProchaineVague()
+getTempsRestantSecondes()
+getProchaineVague()
-declencherVague()
-genererPositionBord()
crée
addUnite()
Ennemi
-HP_INITIAL: int
-VITESSE: float
-DEGATS: int
+Ennemi(int GX, int GY, Terrain terrain)
Terrain
+addUnite(Unite u)
utilise
CommandeDeplacementEnnemi
+executer(Unite unite, double dt) : : boolean
Unite



5.4.1 : Génération automatique des ennemis par vagues

Les ennemis sont générés automatiquement par le `GestionnaireVagues` toutes les 3 minutes. Chaque vague contient un nombre croissant d'ennemis (1 pour la première vague, 2 pour la deuxième, etc.).

5.4.2 : Déplacement des ennemis vers le bâtiment maître

Les ennemis se déplacent en ligne droite vers le bâtiment maître en utilisant des coordonnées continues. La classe `CommandeDeplacementEnnemi` calcule la direction et la distance pour ajuster leur position à chaque tick.

5.4.3 : Actions des ennemis (destruction des bâtiments et unités à portée)

Les ennemis peuvent détruire les bâtiments et unités à portée. Cette fonctionnalité est partiellement implémentée dans `CommandeDeplacementEnnemi` mais nécessite des améliorations pour gérer les collisions et les dégâts.

5.4.4 : Mort des ennemis au contact des unités de défense

Les ennemis meurent au contact des unités de défense. Cette fonctionnalité est prévue mais pas encore implémentée dans le code actuel.

5.5 : Génération du terrain en début de partie

Structures de données principales et constantes

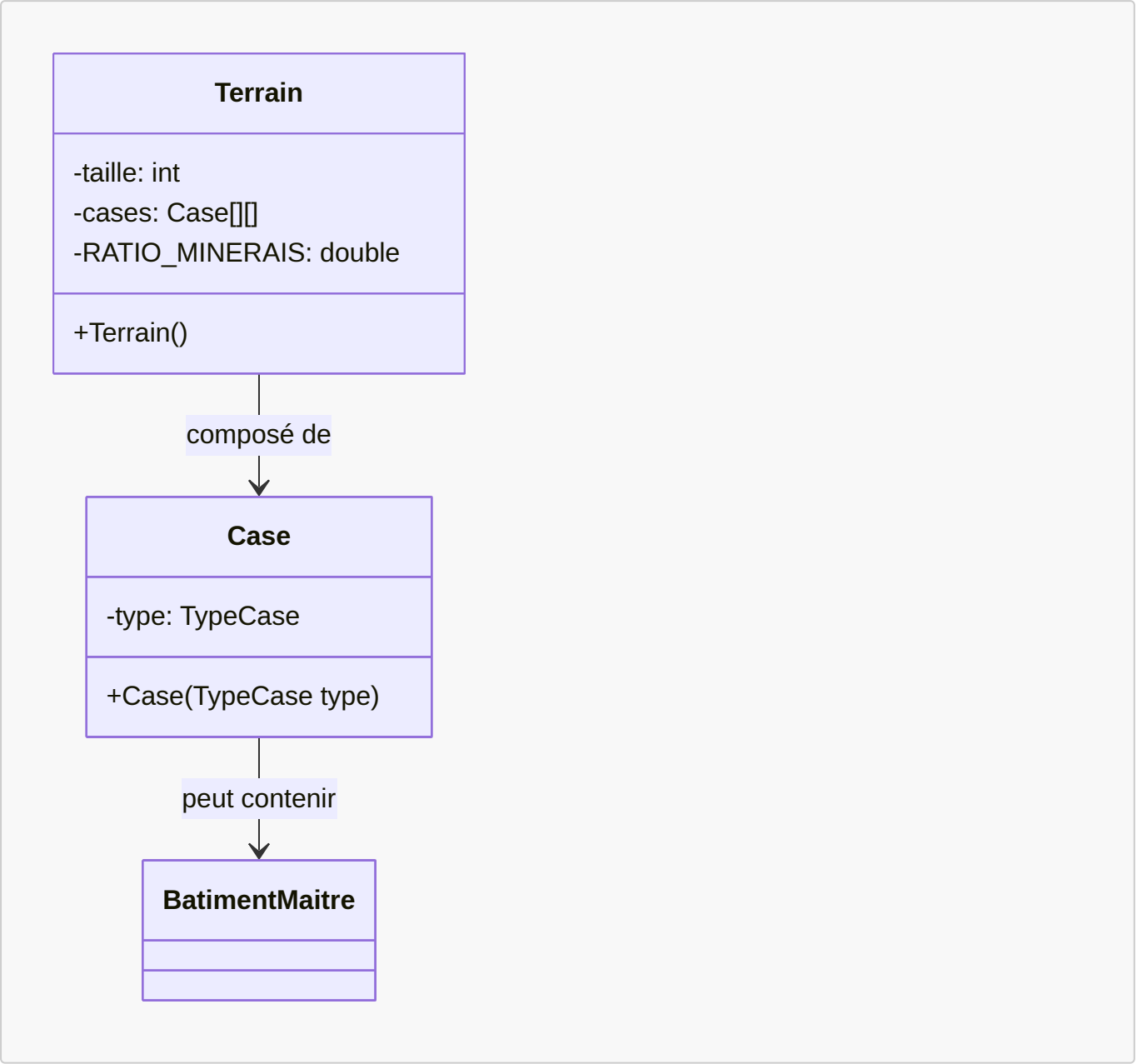
- Classe `Terrain` : représente la grille de jeu, contient les cases et gère la génération du terrain.
- Attributs :
  - `taille` (int) : taille de la grille (ex: 20x20).
  - `cases` (Case[][]) : matrice représentant les cases du terrain.
  - `RATIO_MINERAIS` (double) : pourcentage de cases contenant du minerai (ex: 0.1 pour 10%).
- Méthodes :
  - `Terrain()` : constructeur qui créé le terrain.

Algorithme abstrait

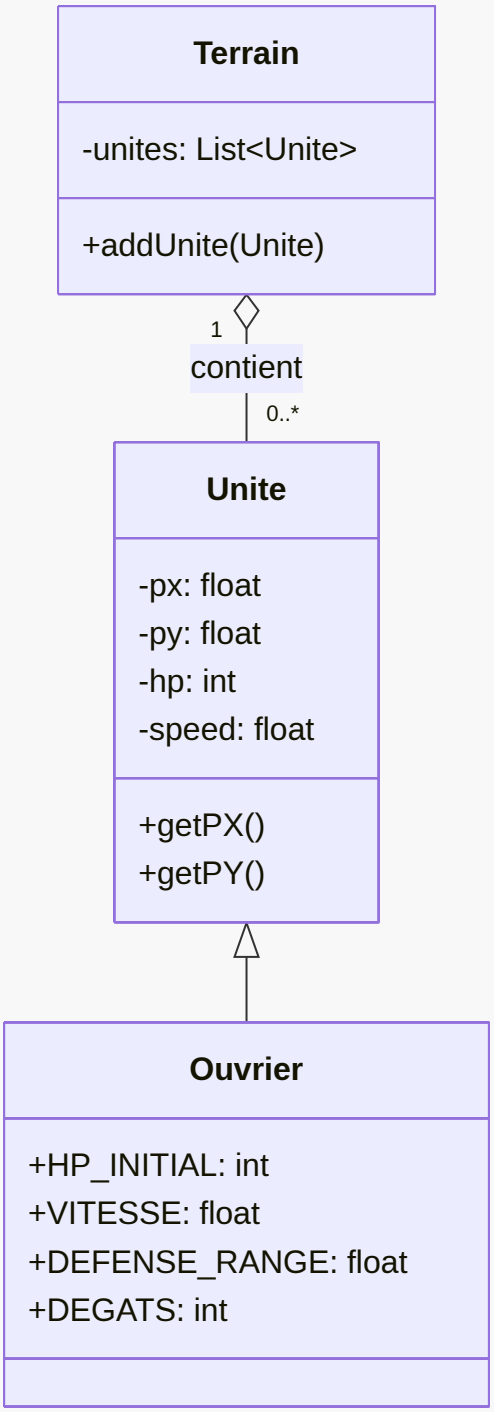
1. Déterminer le nombre de cases contenant un minerai : `nombreMinerais = (int)(taille * taille * RATIO_MINERAIS)`.
2. Générer une liste contenant `nombreMinerais` positions aléatoires différentes les unes des autres et différentes de la position du bâtiment maître.

3. Pour chaque case dans la matrice `cases`, si sa position est contenue dans la liste on y place une case MINERAI, sinon une case VIDE.
4. Crée un nouveau `BatimentMaitre` et le place au centre de la grille.

Diagramme de classe



5.5.3 Création de l'unité de base initiale



Au lancement du jeu dans `Main.main()`, une unité initiale de type `Ouvrier` est automatiquement créée et positionnée sur le terrain. Cette initialisation se fait via la ligne :

```
terrain.addUnite(new Ouvrier(terrainSize/2 + 5, terrainSize/2 + 5,
terrain));
```

L'ouvrier est placé légèrement décalé (5 cases) par rapport au centre du terrain pour éviter de se superposer avec le bâtiment maître. La classe `Ouvrier` étend `Unite` et définit ses caractéristiques :

- **Points de vie initiaux** : `HP_INITIAL = 10`

- **Vitesse de déplacement** : `VITESSE = 0.2f`
- **Portée de défense** : `DEFENSE_RANGE = Case.TAILLE`
- **Dégâts infligés** : `DEGATS = 2`

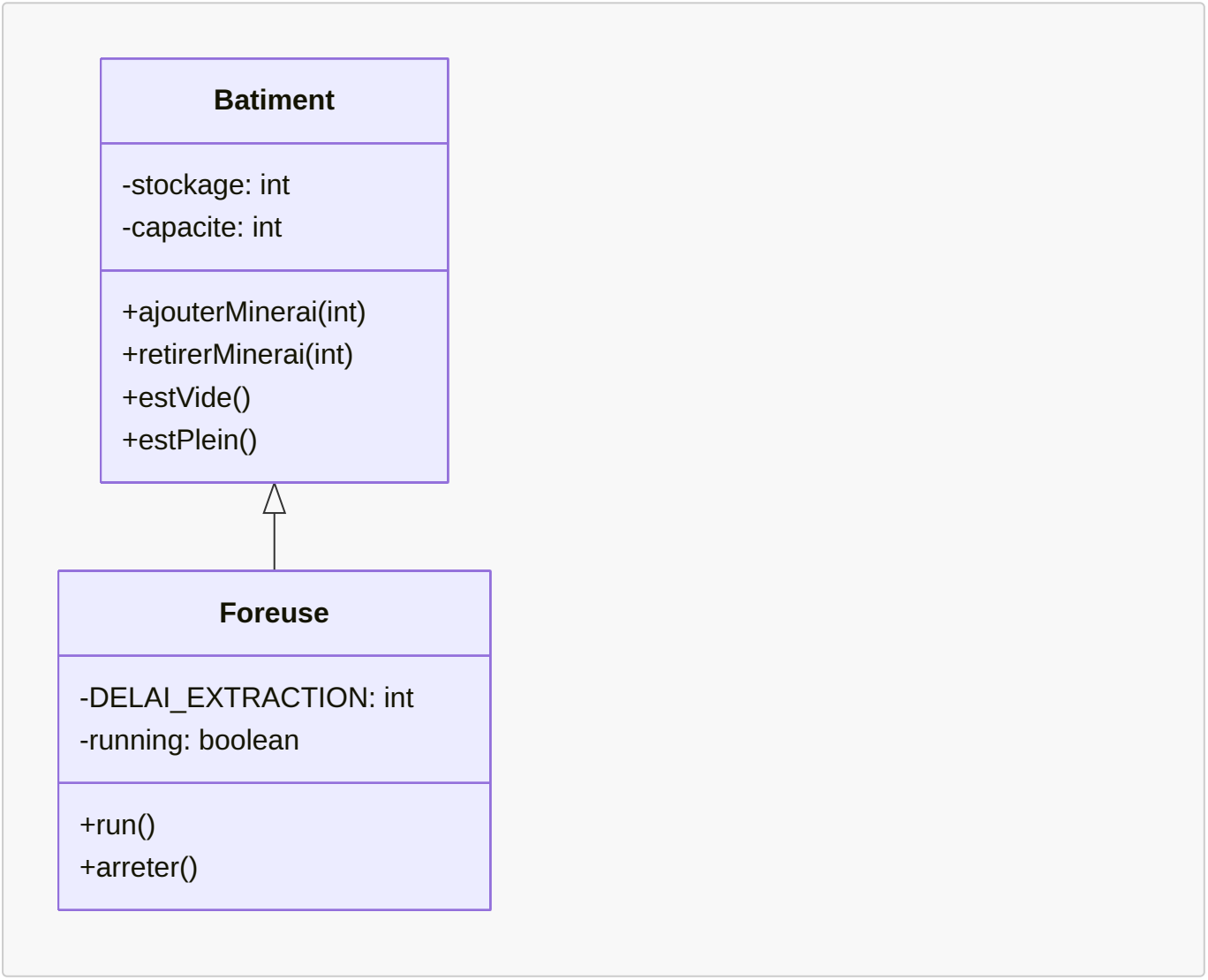
Cette unité permet au joueur de commencer immédiatement à récolter du minerai et à construire des bâtiments.

5.6 : Actions et déplacements des unités

5.6.1 : Miner

La classe `Minerai` modélise le transport d'une unité de ressource sur la carte.

- `Minerai` implémente `Runnable`.
- La position courante est stockée via `x` et `y`.
- Le déplacement s'exécute par pas temporels (`DELAI_TRANSPORT`).
- La **grille** ou le **menu** .



5.6.2 : Construction de bâtiments

TEMPORAIRE : à modifier une fois que les unités seront implémentées (actuellement, la construction est gérée directement via le menu).

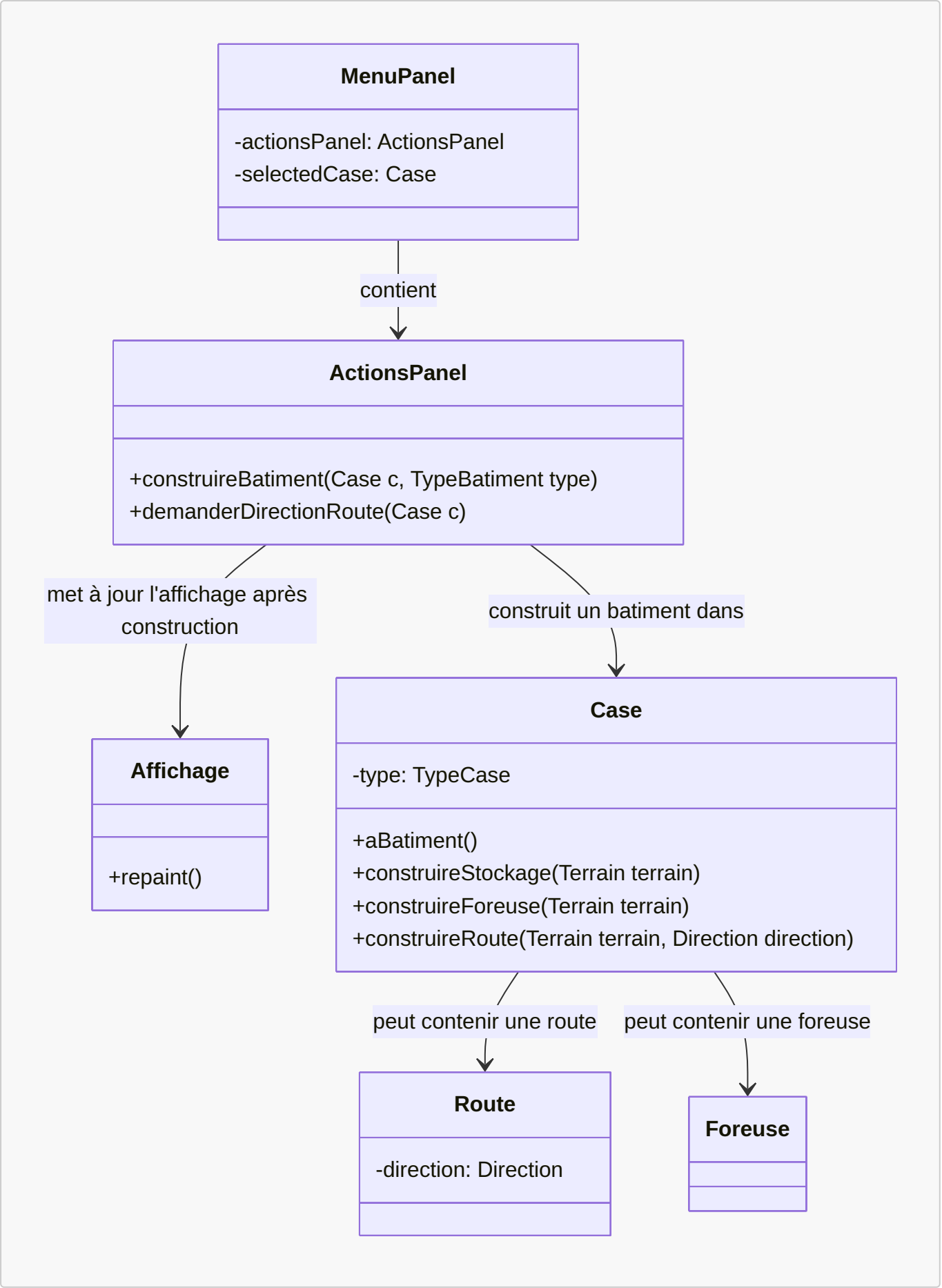
Lorsque l'utilisateur sélectionne une case ne contenant aucun bâtiment, il peut cliquer sur un bouton pour construire un bâtiment (ex: route, foreuse, etc.).

La classe `ActionsPanel` dans `MenuPanel` affiche les boutons pour la construction de bâtiments. Lorsqu'un bouton est cliqué, elle vérifie d'abord que la case sélectionnée `selectedCase` est constructible (pas de bâtiment déjà présent, type de case compatible).

Si l'utilisateur veut construire une route, un `JOptionPane` s'ouvre pour demander la direction. Ensuite, une instance du bâtiment correspondant (`Route`, `Foreuse`, `Stockage`, `Usine`) est créée et ajoutée à la case cible.

Règle importante : un bâtiment n'est utilisable que lorsqu'il est marqué comme terminé (`estFini() == true`). Tant qu'il n'est pas terminé, les actions qui dépendent de ce bâtiment sont bloquées et réévaluées au cycle suivant. Afin de pouvoir marquer un bâtiment comme terminé, il doit d'abord être construit par une unité.

En mode démonstration, les bâtiments posés par le scénario sont explicitement marqués terminés pour que la simulation démarre immédiatement.



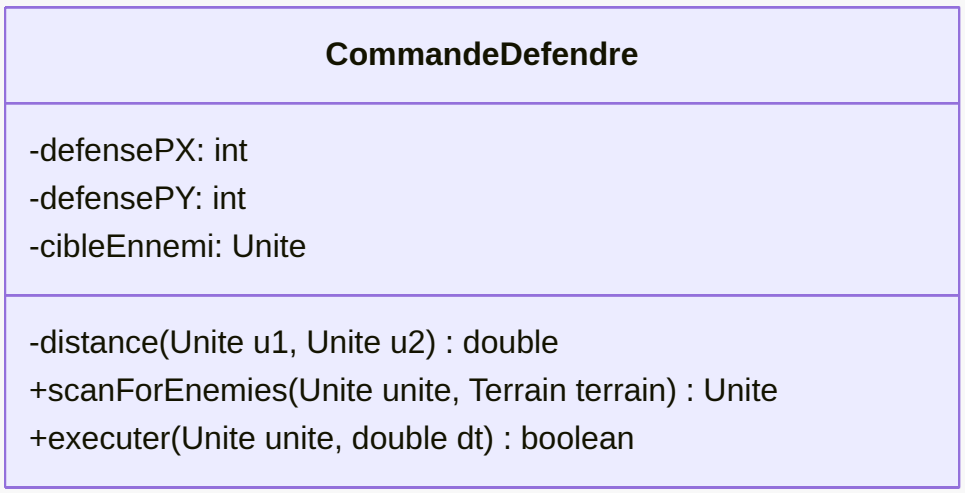
5.6.3 : Transport du minerai

Les unités peuvent transporter du minerai entre les bâtiments. La classe `CommandeDeposit` gère le dépôt de minerai dans le bâtiment maître, tandis que `CommandeMiner` gère la collecte de minerai depuis les gisements.

5.6.4 : Défense

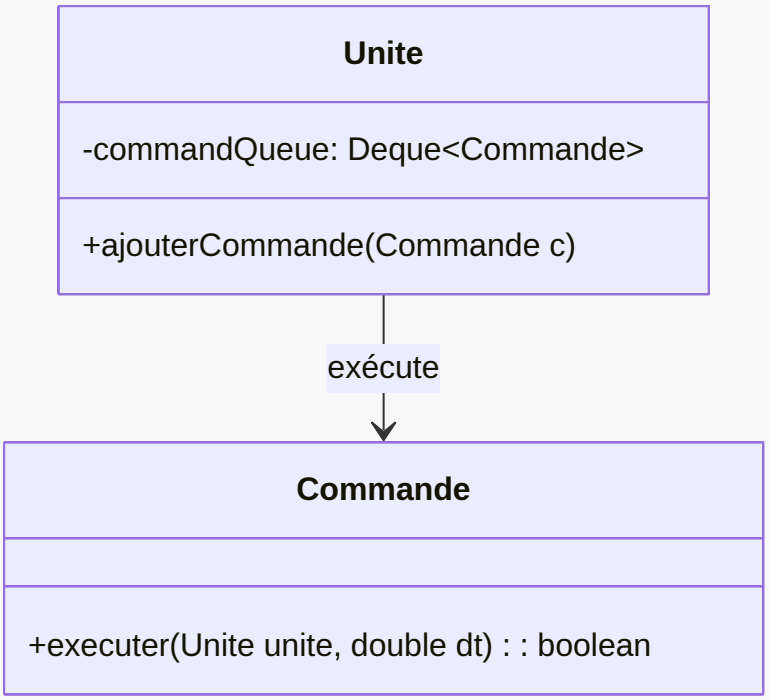
Les unités peuvent être mises en mode défense pour protéger les bâtiments. Lorsqu'une unité est en mode défense, elle attaque automatiquement les ennemis à portée. La classe `CommandeDefendre` gère l'activation du mode défense pour une unité.

Lorsqu'une unité est mise en mode défense par le joueur, elle scan toutes les unités présentes sur le terrain avec `scanForEnemies`. La première unité ennemie détectée dans un rayon définit devient sa cible. Tant que la cible est vivante et à portée, l'unité de défense inflige des dégâts à intervalles réguliers. Si la cible est hors de portée, l'unité se déplace pour la rattraper. Si la cible meurt ou qu'aucune n'est détectée, l'unité se replace à l'endroit qu'elle défendait et se remet à chercher une nouvelle cible.



5.6.5 : Attribution d'ordres aux unités

Les ordres sont attribués aux unités via des commandes (ex: `CommandeMiner`, `CommandeDeplacement`). Ces commandes sont exécutées séquentiellement par l'unité.



5.7 : Menus et interface utilisateur

5.7.1 : Gestion des clics de l'utilisateur

Structures de données principales et constantes

La gestion des interactions utilisateur a été regroupée en plusieurs contrôleurs spécialisés, chacun ayant sa propre responsabilité :

- **Classe `CameraController`**
  Gère les interactions liées au déplacement de la caméra :
  - déplacement via drag souris (clic gauche maintenu)
  - déplacement via clavier (flèches)
  - distinction entre clic et drag grâce à un seuil (`DRAG_THRESHOLD`)
- **Classe `ReactionClic` (implémente `MouseListener`)**
  Gère les clics utilisateur sur la grille :
  - détecte les clics courts (bouton gauche)
  - ignore les événements si un drag est en cours
  - déclenche l'affichage du menu
- **Classe `ReactionHover` (implémente `MouseMotionListener`)**
  Gère le survol des cases :
  - met à jour dynamiquement la case survolée
  - permet un retour visuel en temps réel
- **Classe `Camera`**
  Permet la conversion des coordonnées écran vers la grille :
  - `screenToGridX(int)`
  - `screenToGridY(int)`
    Ces méthodes prennent en compte le décalage de la caméra (offset).

Constantes importantes

- `DRAG_THRESHOLD` : seuil minimal (en pixels) pour considérer un mouvement comme un drag
- `PAN_KEY_SPEED` : vitesse de déplacement de la caméra au clavier

Algorithme abstrait

1. Gestion du clic (ReactionClic)

1. *Lors d'un événement `mouseReleased` :*
    - vérifier que le bouton gauche est utilisé
    - vérifier qu'il ne s'agit pas d'un drag (`cameraController.isDragging()`)
2. *Convertir les coordonnées écran `(x, y)` en coordonnées grille :*

    ```
    (gx, gy) = screenToGrid(x, y)
    ```

3. *Vérifier les bornes :*
    - si hors de la grille → masquer le menu
4. *Récupérer la case correspondante :*
    - si la case contient un bâtiment → afficher le menu
    - sinon → ne rien faire

2. Gestion du drag (CameraController) :

1. `mousePressed` :
    - mémorise la position initiale de la souris
    - mémorise l'offset initial de la caméra
2. `mouseDragged` :
    - calcule le déplacement `(dx, dy)`
    - si dépasse `DRAG_THRESHOLD` → activation du mode drag
    - met à jour la caméra :

    ```
    offset = offset_initial - déplacement_souris
    ```

3. `mouseReleased` :
    - désactive le drag

3. Gestion du survol (ReactionHover)

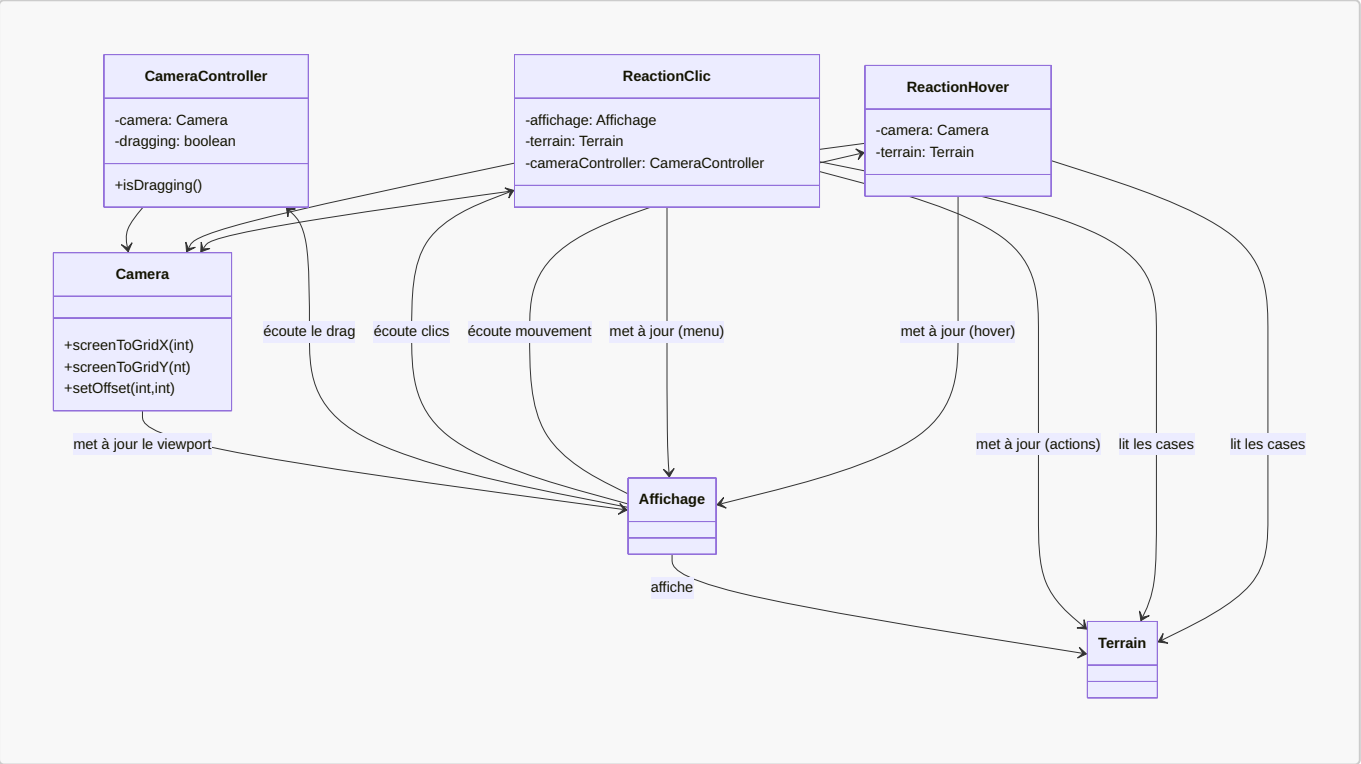
1. À chaque mouvement souris : conversion écran → grille
2. Vérification des bornes : si hors grille → suppression du survol
3. Mise à jour uniquement si la case change : optimisation pour éviter des redessins inutiles

4. Conditions limites à respecter

- Différenciation clic / drag Un déplacement léger ne doit pas être interprété comme un drag.
- Conversion correcte des coordonnées Les coordonnées doivent toujours prendre en compte l'offset de la caméra.
- Respect des bornes de la grille

  0 ≤ gx < taille 0 ≤ gy < taille
- Focus clavier : nécessaire pour capter les événements clavier `requestFocusInWindow()`

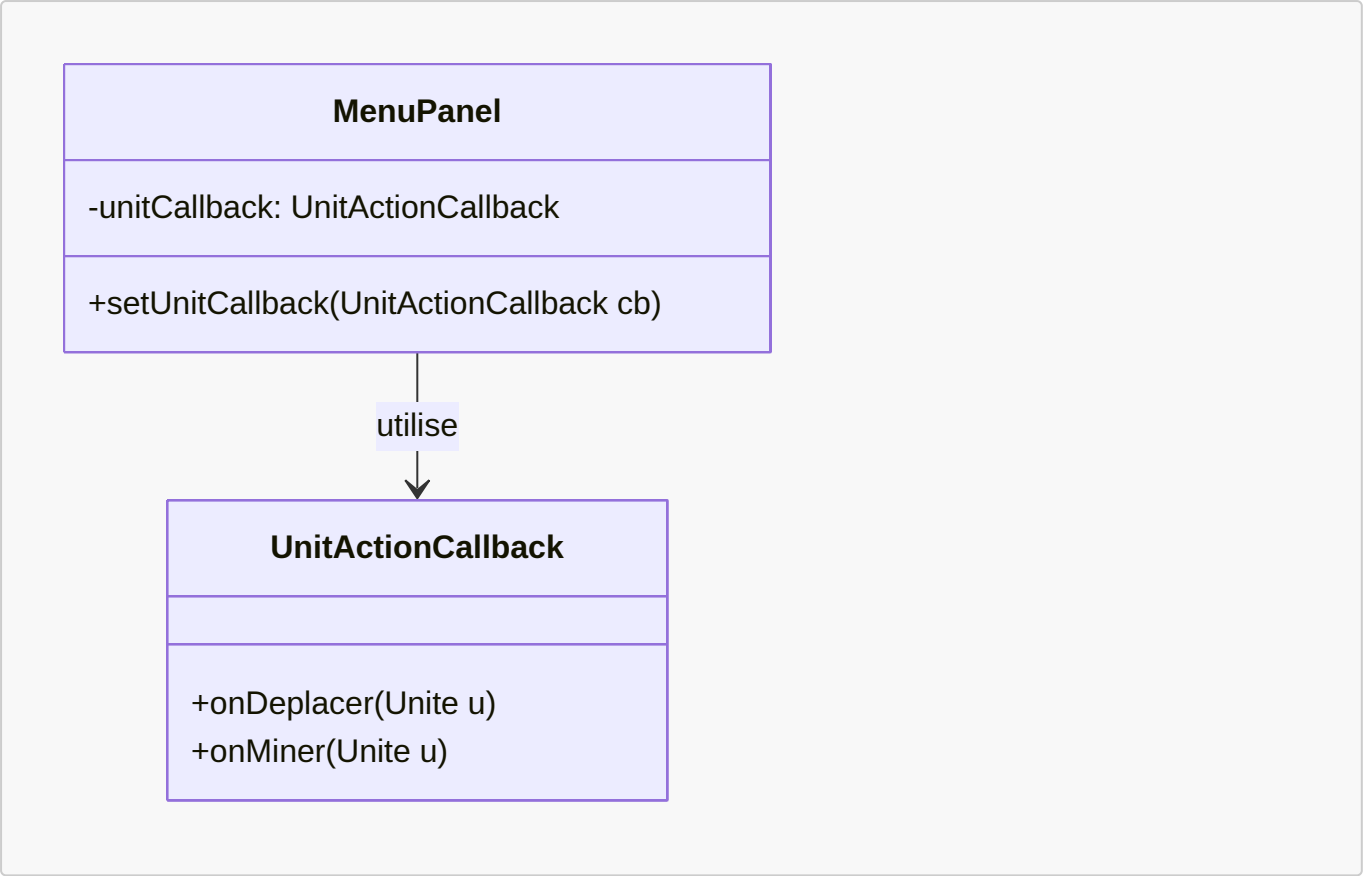
Diagramme de classes simplifié



Ce diagramme représente l'essentiel de la logique du jeu et suit le patron **MVC** (Model-View-Controller). Le modèle (`Terrain`) contient les données du jeu, la vue (`Affichage`) gère l'affichage en fonction du modèle, et les contrôleur (`ReactionClic` , `ReactionHover` , `CameraController` et `Camera`) gère le modèle tout en mettant également à jour directement la vue pour fournir un retour visuel.

5.7.2 : Menu pour les unités

Le `MenuPanel` affiche les actions disponibles pour les unités sélectionnées. Il permet de donner des ordres comme déplacer, miner, construire, ou défendre. Les actions sont gérées via des callbacks définis dans `UnitActionCallback`.



5.7.3 : Menu des bâtiments (affichage des quantités de minerai)

Structures de données principales et constantes

- **Classe `MenuPanel`** : panneau principal affiché à droite de l'écran qui contient :
  - `HeaderPanel`
  - `StatsPanel`
  - `ActionsPanel`
- **Classe `StatsPanel`** : responsable de l'affichage des informations du bâtiment:
  - une barre de capacité (`CapacityBar`)
  - des labels pour les valeurs numériques
- **Classe `CapacityBar`**
  - composant graphique personnalisé
  - affiche le ratio de remplissage du stockage

Algorithme abstrait

1. Affichage du menu

1. Lors d'un clic sur une case contenant un bâtiment :
   - appel à `showMenu(c)`
2. Le menu :
   - devient visible
   - stocke la case sélectionnée
   - déclenche un rafraîchissement (`refresh()`)

2. Construction des statistiques

Si la case sélectionnée change :

- suppression des composants existants
- reconstruction complète de l'interface :
  - création de la barre de capacité
  - création des lignes d'information

3. Mise à jour dynamique (optimisation)

Si la case sélectionnée reste la même :

1. Ne pas reconstruire l'interface
2. Mettre à jour uniquement :
   - la barre (`CapacityBar.updateValues`)
   - les labels

   Cela permet d'éviter des recalculs inutiles, des re-layout coûteux et des effets visuels indésirables (en rafraîchissant constamment le menu)

4. Mise à jour de la barre de capacité

1. *Calcul du ratio :*

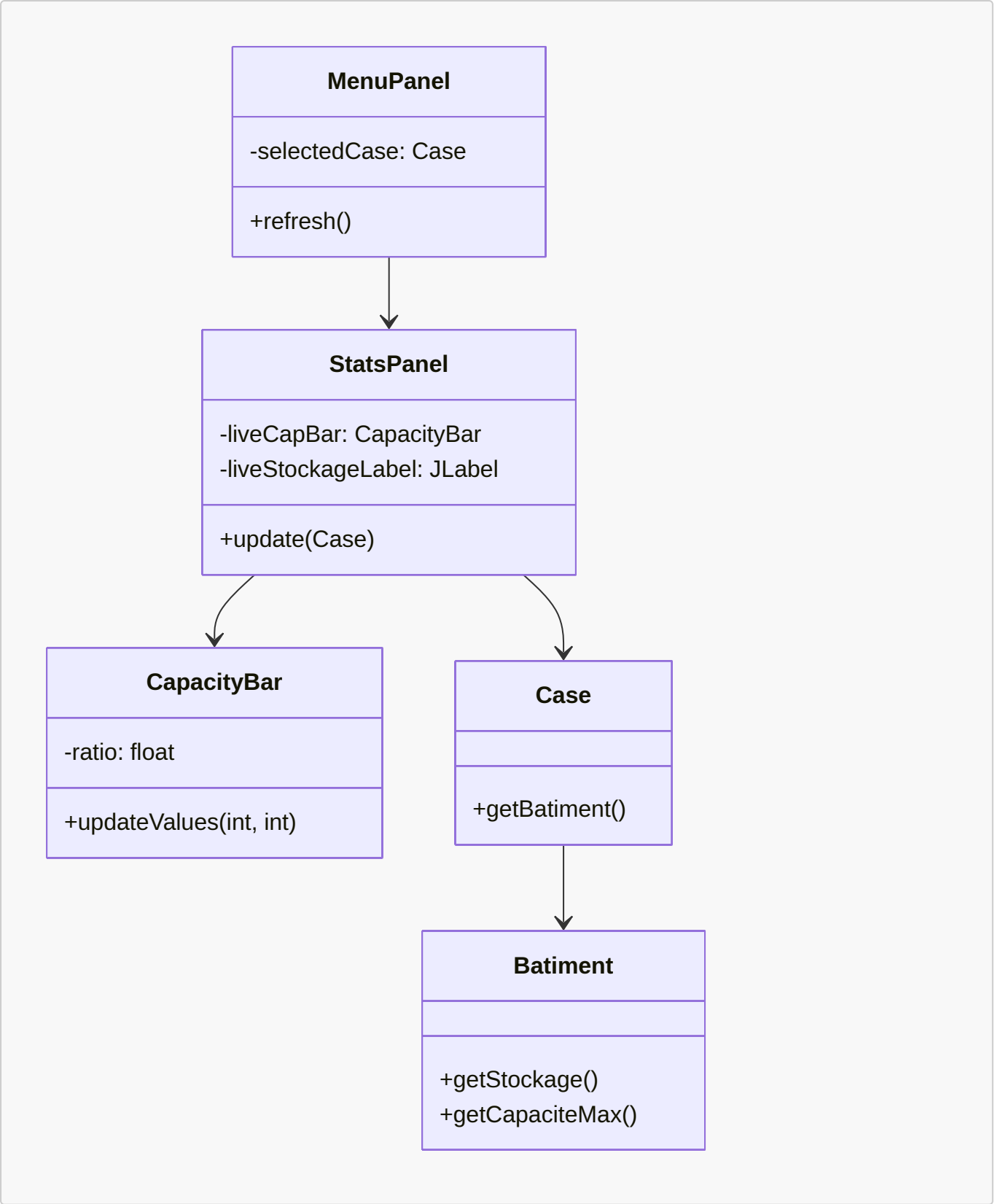
   ratio = stockage / capacité

2. *Choix de la couleur :*
   - vert : faible remplissage
   - orange : moyen
   - rouge : proche de la saturation
3. *Redessin uniquement si les valeurs changent*

Conditions limites à respecter

- **Capacité nulle** :éviter la division par zéro
- **Respect de l'invariant** `stockage ≤ capacité`
- **Performance** : ne pas recréer l'interface à chaque mise à jour (sauf en cas d'absolue nécessité)
- **Cohérence avec les threads** : les valeurs peuvent changer de manière asynchrone (foreuse, transport)

Diagramme de classes simplifié



5.7.4 : Vue d'ensemble des données (minerais, unités, bâtiments) (optionnel)

Cette fonctionnalité n'a pas été implémentée dans la version actuelle. Cependant, l'interface actuelle permet déjà de visualiser :

- Les bâtiments et leurs états (normal, en construction) via le menu contextuel à droite
- La case sélectionnée avec un encadrement orange
- La case survolée avec un surlignage blanc
- L'écran de Game Over quand le bâtiment maître est détruit

Une future version pourrait inclure un panneau d overview affichant :

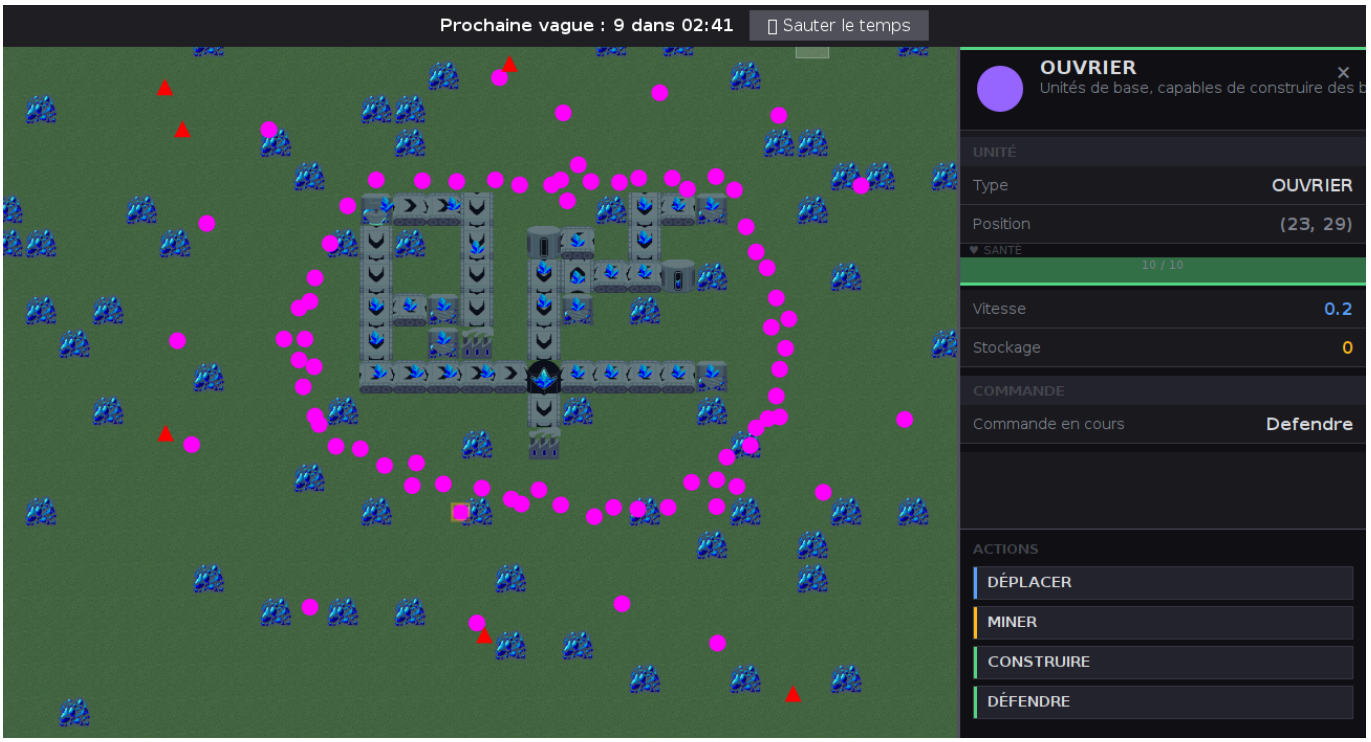
- Le nombre total de minerais collectés
- Le nombre d'unités actives
- L'état de tous les bâtiments (HP, production, stockage)

6. Résultats

Le projet a permis de développer un jeu de stratégie solo en temps réel avec les fonctionnalités suivantes :

- Gestion des coordonnées pour les bâtiments et les unités.
- Affichage des objets (unités, minerais, bâtiments).
- Gestion des bâtiments (usine, stockage, mine, routes).
- Actions et déplacements des unités (miner, construire, transporter).
- Menus et interface utilisateur pour interagir avec le jeu.

Prochaine vague : 1 dans 01:28
Sauter le temps
OUVRIER
Unités de base, capables de construire des b
UNITÉ
Type
OUVRIER
Position
(30, 25)
SANTÉ
10 / 10
Vitesse
0.2
Stockage
2
COMMANDE
Commande en cours
Construire
ACTIONS
DÉPLACER
MINER
CONSTRUIRE
DÉFENDRE


Prochaine vague : 9 dans 02:41
Sauter le temps
OUVRIER
Unités de base, capables de construire des b
UNITÉ
Type
OUVRIER
Position
(23, 29)
SANTÉ
10 / 10
Vitesse
0.2
Stockage
0
COMMANDE
Commande en cours
Defendre
ACTIONS
DÉPLACER
MINER
CONSTRUIRE
DÉFENDRE

Validation et Tests

La classe `Validation` permet de valider les arguments et les états du jeu. Elle est configurable via une propriété JVM (`-Dpcii.validation.strict=false`).

Des tests unitaires ont été implémentés pour valider les fonctionnalités critiques :

- `ForeuseThreadTest` : Vérifie que la foreuse extrait correctement du minerai.
- `MineraiTest` : Vérifie le transport du minerai.
- `ReactionClicTest` : Vérifie la gestion des clics utilisateur.
- `TestGameOver` : Teste la fonctionnalité de fin de partie en simulant des attaques sur le bâtiment maître jusqu'à sa destruction et l'affichage de l'écran Game Over.
- Les vagues d'ennemis peuvent être testées en exécutant le jeu principal et en observant le compteur en haut de l'écran. Le bouton "Sauter le temps" permet de tester rapidement plusieurs vagues.

Ces tests garantissent la robustesse du code et facilitent la maintenance.

7. Documentation utilisateur

Prérequis

- Java JDK 17 ou supérieur.
- Un environnement de développement (ex: IntelliJ IDEA, VS Code).

Installation

1. Cloner le dépôt GitHub.

2. Depuis le répertoire racine, compiler le projet avec la commande :

```
javac -d target/classes $(find src -name "*.java")
```

3. Exécuter le jeu avec la commande :

```
java -cp target/classes main.Main
```

Utilisation

Contrôles de base

Action	Contrôle
Sélectionner une case/bâtiment/unité	Clic gauche
Déplacer la caméra	Clic gauche + glisser
Zoomer/Dézoomer	Molette de la souris ou touches +/-
Ouvrir le menu d'actions	Clic gauche sur une case sélectionnée

---

Comment jouer ?

1. Récolter du minerai

- Sélectionnez un **Ouvrier**
- Déplacez-le sur un **filon de minerai** (case grise)
- Cliquez sur **"Miner"** dans le menu
- L'Ouvrier remplit son stockage (max 5 minerais)
- Déplacez-le vers un **Bâtiment Maître** ou **Stockage** et cliquez sur **"Déposer"**

2. Construire des bâtiments

- Sélectionnez une **case vide**
- Dans le menu, choisissez un bâtiment :
  - **Foreuse** (10 minerais) :
    - **Rôle** : Extraire **automatiquement** du minerai d'un filon
    - **Fonctionnement** :
      - Doit être posée **sur une case avec du minerai** (sinon ne produit rien)
      - Extraire **1 minerai toutes les 2 secondes**
      - **Stockage limité à 1 minerai** → doit être vidée régulièrement
      - **Automatisation** : Connectez-la avec des **Routes** pour acheminer le minerai vers un Stockage ou le Bâtiment Maître
    - **Astuce** : Plus vous avez de Foreuses, plus vous récoltez rapidement !
  - **Route** (1 minerai) : Transport du minerai (choisir une direction ↑↓←→)
    - **Rôle** : Permet de déplacer automatiquement le minerai d'un bâtiment à un autre
    - **Fonctionnement** :

            - Le minerai suit la direction indiquée (ex: une route vers la droite → le minerai se déplace vers la case à droite)
            - **Un seul minerai à la fois** : si une route est déjà occupée, le minerai attend que la route soit libre
            - **Automatisation** : Connectez les routes entre les Foreuses, les Stockages et le Bâtiment Maître pour créer un réseau de transport efficace
        - **Astuce** : Planifiez bien vos routes pour éviter les embouteillages !
    - **Stockage** (10 minerais) : Stocke jusqu'à 10 minerais
    - **Usine** (50 minerais) : Produit des Ouvriers automatiquement
- Cliquez sur un **Ouvrier** et choisissez **"Construire"** pour construire le bâtiment à l'emplacement sélectionné

3. Se défendre

- Sélectionnez un **Ouvrier**
- Cliquez sur **"Défendre"**
- Cliquez sur une **case à défendre** (ex: autour du bâtiment maître)
- L'Ouvrier attaquera automatiquement les ennemis à proximité (portée : 2 cases)

Astuces

- **Vagues d'ennemis** : Arrivent toutes les 3 minutes. Utilisez le bouton **"Sauter le temps"** en haut pour tester rapidement.
- **Direction des Routes** : Le minerai suit la flèche. Vérifiez le sens !
- **Bâtiment Maître** : **À protéger absolument** (100 PV). Sa destruction = Game Over.

8. Documentation développeur

Classes principales

Voici une liste des classes principales du projet, organisées par package et rôle qui pouvent être intéressantes à regarder en priorité pour comprendre le fonctionnement du jeu :

- `Main` : Point d'entrée du jeu.
- `AsyncExecutor` : Gère l'exécution des threads pour les unités et les bâtiments.

Model

- `Terrain` : Classe centrale du modèle, représente la grille de jeu et contient les cases, les unités et les bâtiments.

- `Unite` : Classe de base pour `Ouvrier` et `Ennemi`. Gère les unités et leurs commandes. On peut ajouter des nouvelles types d'unités en créant une nouvelle classe qui étend `Unite` et en définissant ses caractéristiques (HP, vitesse, etc.) et ses comportements (commandes) spécifiques (ex: attaque, défense, etc.).

- Dossier `commande` : Contient les différentes commandes pour les unités (miner, déplacer, défendre, etc.). Si vous voulez ajouter une nouvelle action pour les unités, c'est ici que vous devez créer une nouvelle classe de commande et il faut respecter l'interface `Commande` (implémenter la méthode

`executer(Unite u, double dt)` qui s'effectue chaque tick (`dt`) et retourne un boolean indiquant si la commande est terminée ou non).

- `Batiment` : Classe de base pour tous les bâtiments. Les bâtiments spécifiques (Foreuse, Route, Stockage, Usine) étendent cette classe et définissent leurs propres caractéristiques et comportements.

View

- `Fenetre` : Gère la fenêtre principale du jeu.
- `Affichage` : Gère le rendu graphique du terrain et des entités.
- `AffichageTerrain` : Gère l'affichage spécifique du terrain et des cases.
- `MenuPanel` : Gère l'interface utilisateur et les actions.
- `Camera` : Gère la conversion des coordonnées écran <-> grille.

Controller

- `CameraController` : Gère les interactions liées à la caméra (drag, clavier).
- `ReactionClic` : Gère les clics de l'utilisateur pour sélectionner des cases et afficher le menu.
- `ReactionHover` : Gère le survol des cases pour un retour visuel en temps réel.

Constantes modifiables

Bâtiments

- `Foreuse.DELAI_EXTRACTION_MS` : Délai entre deux extractions de minerai.
- `Minerai.DELAI_TRANSPORT_MS` : Délai de transport du minerai.
- `Usine.DELAI_PRODUCTION_MS` : Délai de production des unités.
- `Foreuse.COUT_CONSTRUCTION` : Coût de construction d'une foreuse.
- `Route.COUT_CONSTRUCTION` : Coût de construction d'une route.
- `Stockage.COUT_CONSTRUCTION` : Coût de construction d'un stockage.
- `Usine.COUT_CONSTRUCTION` : Coût de construction d'une usine.
- `Stockage.CAPACITE` : Capacité maximale de stockage d'un stockage.
- `Usine.CAPACITE` : Capacité maximale de stockage d'une usine.
- `BâtimentMaitre.CAPACITE` : Capacité maximale du bâtiment maître.
- `Usine.COUT_PRODUCTION` : Coût de production d'une unité dans l'usine.

Unités

- `Ennemi.VITESSE` : Vitesse de déplacement des ennemis.
- `Ennemi.HP_INITIAL` : Points de vie initiaux des ennemis.
- `Ennemi.DEGATS` : Dégâts infligés par les ennemis.

- `Ouvrier.VITESSE` : Vitesse de déplacement des ouvriers.
- `Ouvrier.HP_INITIAL` : Points de vie initiaux des ouvriers.
- `Ouvrier.DEFENSE_RANGE` : Portée de défense des ouvriers.
- `Ouvrier.DEGATS` : Dégâts infligés par les ouvriers en mode défense.
- `GestionnaireVagues.DELAI_ENTRE_VAGUES_MS` : Délai entre chaque vague d'ennemis.

Fonctionnalités à implémenter

Voici quelques fonctionnalités supplémentaires qui pourraient être implémentées pour améliorer le jeu :

- **Visuels** : Améliorer les graphismes et l'interface utilisateur pour une meilleure expérience de jeu. Par exemple, ajouter une vue d'ensemble des ressources, des unités et des bâtiments: on peut le faire en ajouter les lignes des infos supplémentaires dans le `StatsPanel` (dans `MenuPanel`) pour BâtimentMaitre.
- **Menu de pause/début de jeu** : Ajouter un menu de pause avec des options pour reprendre, redémarrer ou quitter le jeu. On peut le faire en ajoutant un listener pour la touche "P" (ou "Esc") dans le controlleur. Ensuite, tous les threads (unités, foreuses, vagues) doivent être mis en pause et on affiche un menu de pause avec des boutons pour reprendre, redémarrer ou quitter.
- **Contrôle des unités en groupe**: Permettre au joueur de sélectionner plusieurs unités en même temps et de leur donner des ordres groupés (ex: déplacer un groupe d'ouvriers vers une zone, les mettre tous en mode défense, etc.). On peut implémenter cela en ajoutant une fonctionnalité de sélection multiple dans `ReactionClic` (ex: clic droit + glisser pour dessiner un rectangle de sélection). Ensuite, les commandes données à une unité sélectionnée sont appliquées à toutes les unités du groupe.
- **Options de difficulté/settings du jeu** : Ajouter différents niveaux de difficulté (ou des paramètres personnalisables) pour ajuster la vitesse des ennemis, la fréquence des vagues, les ressources disponibles, etc. On peut le faire en ajoutant un menu de configuration accessible depuis l'écran de début de jeu et quand on lance le jeu, on lit les paramètres choisis pour ajuster les constantes abordées auparavant (ex: `Ennemi.VITESSE`, `GestionnaireVagues.DELAI_ENTRE_VAGUES_MS`, etc.).
- **Système de sauvegarde** : Permettre aux joueurs de sauvegarder et charger leur progression. On peut l'implementer en convertissant l'état du jeu (les infos sur toutes les cases sur le terrain et aussi des unités) en un format sérialisé (ex: JSON) et en le sauvegardant dans un fichier. Ensuite, on peut ajouter des options dans le menu pour sauvegarder et charger une partie.

9. Conclusion et perspectives

Réalisation

Nous avons développé un jeu de stratégie solo en temps réel avec une architecture MVC. Les principales fonctionnalités incluent la gestion des bâtiments, des unités, et des ressources, ainsi qu'une interface utilisateur interactive.

Difficultés et solutions

- **Gestion des threads** : Utilisation de `volatile` et de mécanismes d'interruption pour gérer les threads de manière sécurisée.
- **Synchronisation** : Utilisation de files de commandes pour gérer les actions des unités de manière séquentielle.
- **Affichage** : Utilisation de `Graphics2D` pour dessiner les entités et optimisation des performances avec des caches d'images.

Apprentissage

La réalisation de ce projet nous a permis de développer plusieurs compétences clés en programmation concurrente et en développement d'interfaces interactives :

- **Programmation concurrente** : Maîtrise des threads en Java, utilisation des flags `volatile`, gestion des interruptions et synchronisation entre processus concurrents.
- **Architecture MVC** : Séparation claire entre le modèle (.logique métier), la vue (affichage graphique) et le contrôleur (gestion des interactions), appliquée à un jeu en temps réel.
- **Gestion des interactions utilisateur** : Implémentation d'une interface graphique réactive avec Swing, gestion des événements clavier et souris, mise à jour dynamique de l'affichage.
- **Pattern de conception** : Utilisation du pattern Command pour encapsuler les actions des unités dans des objets, permettant une exécution séquentielle et facile à étendre.
- **Gestion de projet** : Travail collaboratif avec Git, organisation des tâches, résolution de bugs complexes liés à la synchronisation.
- **Modélisation objet** : Conception d'une hiérarchie de classes cohérente pour représenter les entités du jeu (unités, bâtiments, terrain) et leurs interactions.

Perspectives

Plusieurs axes d'amélioration ont été identifiés pour les versions futures du projet :

- **Système de niveaux** : Ajouter des niveaux de difficulté progressifs avec des configurations de terrain, de ressources et d'ennemis différentes.
- **Sauvegarde/chargement** : Implémenter un système de sauvegarde de la partie (sérialisation) pour permettre aux joueurs de reprendre une partie ultérieurement.
- **Améliorations graphiques** : Intégrer des sprites animés pour les unités et bâtiments, ajouter des effets visuels (particules, animations fluides), implémenter un système de caméra raisonnable.
- **Synchronisation avancée** : Utiliser des verrous (`ReentrantLock`) ou des sémaphores pour protéger les accès concurrents aux ressources partagées (ex: stockage des foreuses, liste des unités).
- **Équilibrage du jeu** : Ajuster les paramètres (vitesse, dégâts, coûts) pour un gameplay plus équilibré et amusant.
- **Nouveaux types d'unités** : Implémenter des unités spécialisées avec des capacités uniques (ex: soldats avec portées étendues, collecteurs plus rapides).
- **Interface utilisateur améliorée** : Ajouter un menu de configuration, des indicateurs visuels (barres de vie, de stockage), un système d'aide contextuelle.
- **Mode multijoueur** : Étendre l'architecture pour supporter le jeu en réseau avec plusieurs joueurs contrôlant chacun leur colonie.
- **Intelligence artificielle** : Améliorer le comportement des ennemis avec des stratégies plus sophistiquées (ex: attaque coordonnée, ciblage prioritaire).

- **Tests automatisés** : Ajouter une suite de tests unitaires pour valider la logique métier et prévenir les régressions.